姓名：闻子博　　学号：20232444

《GNSS 定位新技术及数据处理方法》课后大作业（一）

GNSS 精密单点定位

任务部分：在《卫星导航定位原理》单点定位程序基础上，参考 PPPH 算法或 RTKLIB 算法，实现单点定位精度数量级的提升（到分米级或厘米级）。

选做部分：实现 mm 的静态或动态精密单点定位。

一、 算法

1.1 背景

精密单点定位技术（Precise Point Positioning，PPP）是上世纪九十年代末期发展起来的一种绝对定位技术。在 PPP 之前，绝对定位家族里大都靠 SPP 去维持其发展。SPP 就是标准单点定位技术，也称为伪距单点定位技术。其基本定位原理也是基于最小二乘，但是不涉及载波相位观测值，因此无需考虑模糊度固定的问题。但是伪距观测值的噪声很大，所以定位精度不是很高。而载波相位观测值的噪声小，相对于伪距观测值有先天性的优势。随着 PPP 技术的提出，从刚开始的 PPP 浮点解到现如今的 PPP 固定解，PPP 技术带来了 GNSS 历史上的一次技术革命。PPP 技术利用高精度的载波相位观测值与伪距观测值以及一系列精密产品，便可完成单台接收机的高精度定位。就目前来说，PPP 技术已经相对成熟，但是仍然具有如快速模糊度固定算法、实时动态 PPP 等方面的挑战。

1.2 PPP 基本算法/原理

1.2.1 无电离层组合模型

无电离层组合模型利用不同频率信号间的组合消除电离层延迟的低阶项。也是目前双频、三频信号最常用的组合方式。其数学模型如下：

$$P_{IF} = \frac{{f_1}^2}{{f_1}^2 - {F_2}^2} P_1 + \frac{-{f_2}^2}{{f_1}^2 - {f_2}^2} P_2$$

$$L_{IF} = \frac{{f_1}^2}{{f_1}^2 - {F_2}^2} L_1 + \frac{-{f_2}^2}{{f_1}^2 - {f_2}^2} L_2$$

新组合形成组合后的伪距和载波观测值，模糊度也是组合后的无电离层组合模糊度。因此无法计算原始 L1 和 L2 频率的模糊度。

1.2.2 UoFc 模型

Uofc 模型与无电离层组合模型相似，同样消除了电离层延迟项。但是其运用的是伪距和载波相位之间电离层项大小相等符号相反的性质。因此，其又被称为半和模型。其数学模型如下：

$$P_{IF,P1} = \frac{1}{2}(P_1 + L_1)$$

$$P_{IF,P2} = \frac{1}{2}(P_2 + L_2)$$

与无电离层组合相比，其最大的优势就在于可以进行单频的定位。除此之外，半和模型组合后的噪声减半，且可以计算原始观测值L上的模糊度。

1.2.3 非差非组合模型

无电离层组合和半和模型虽然消除了电离层延迟，但是其造成了观测信息的损失。如电离层的时空相关性等等。其次，无电离层组合模型放大了观测噪声。非差非组合模型从原始观测方程出发进行的，即不进行任何组合的模型。其函数模型如下：

$$P_1 = \rho + c\cdot(dt - dT) + T + I_{L_1} + dm_{P_1} + \varepsilon_{P_1}$$

$$P_2 = \rho + c\cdot(dt - dT) + T + \gamma I_{L_1} + dm_{P_2} + \varepsilon_{P_2}$$

$$L_1 = \rho + c\cdot(dt - dT) + T - I_{L_1} + \lambda_1 B_1 + \delta m_{L_1} + \varepsilon_{L_1}$$

$$L_2 = \rho + c\cdot(dt - dT) + T - \gamma I_{L_1} + \lambda_2 B_2 + \delta m_{L_2} + \varepsilon_{L_1}$$

其与无电离层组合相比，优势在于保留了更多的观测信息，可以利用高电离层约束，使模糊度固定效率更高。并且已有研究证明，非差非组合与半和模型是等价的，且都优于无电离层组合。

1.2.4 PPP 精密钟差产品

在用户端，PPP 常用的精密钟差产品都来源于 IGS 发布的产品。但是IGS的产品在估计钟差时，同样使用的是无电离层组合，因此其在精密产品中，除了包含真实的卫星钟差外，还包含有无电离层组合的硬件延迟偏差。这就涉及了无电离层组合在进行参数估计时的误差处理策略。如常见的硬件延迟等。在精密钟差中包含的硬件延迟如下：

$$\frac{{f_1}^2}{{f_1}^2 - {f_2}^2} {b_p}_1^s + \frac{-{f_2}^2}{{f_1}^2 - {f_2}^2} {b_p}_2^s$$

其中 b 为卫星端 P1 和 P2 上的硬件延迟。

因此在使用 IGS 的精密钟差产品时，必须考虑这项误差。除此之外，精密钟差产品中还包含有参考基准误差，该项误差吸收进了接收机钟差。

1.2.5 参数估计

(1)无电离层组合

对于无电离层组合观测值，首先通过三种 DCB 产品（P1-P2，C1-P1，C2-P2），将其中一个伪距观测值改为 P1/P2。因为无电离层组合观测值使用的观测值就是 P1 和 P2，如果出现的观测值是 C1 和 C2，需要使用 DCB 改正为 P1 或者 P2。

组合后无电离层组合伪距观测值中包含有卫星端硬件延迟，但是精密钟差中也包含有卫星端硬件延迟，因此他们相互抵消了。而接收机端无电离层组合硬件延迟则吸收进接收机钟差。

无电离层组合载波观测值中，精密钟差引入的卫星端硬件延迟被模糊度吸收，而接收机端硬件延迟则被接收机钟差吸收。这也是为什么无电离层组合模糊度损失整周特性的原因之一。

因此，在无电离层组合观测值中需要估计的参数包括，接收机三维位置、接收机钟差、天顶对流层延迟、模糊度。

(2)非差非组合

对于非差非组合模型，也需要使用三种 DCB 产品将伪距观测值改为 P1/P2 观测值。

对于精密钟差中引入的无电离层卫星端硬件延迟，在伪距观测值中被电离层参数吸收，在载波观测值中则被电离层和模糊度吸收。

接收机端硬件延迟则被接收机钟差吸收。

因此对于非差非组合模型，需要估计的参数包括：接收机三维坐标，接收机钟差，电离层延迟，天顶对流层延迟，模糊度等。

1.2.6 PPP 中的易混淆名词

IFB、ISB

IFB：伪距、载波频间偏差。

主要针对的是一 FDMA 技术的卫星，如俄罗斯的 GLONASS 卫星。不同卫星一般采用不同的频率，因此需要引入 IFB 参数估计。

ISB：系统间偏差

主要针对多系统，可以解释为GNSS时间基准之间的偏差（如BDS与 GPS 时的差异）与不同 GNSS 系统对应接收机伪距硬件延迟差异的综合性偏差。

差分码偏差和码偏差

码偏差：又称硬件延迟，是指GNSS信号在卫星和接收机端硬件内部的时间延迟。

对于卫星端硬件延迟，可以理解为GNSS信号从产生到天线相位中心的延迟。

对于接收机端硬件延迟，可以理解为GNSS信号从接收天线相位中心到信号捕获跟踪的时间延迟。

1.2.7 无电离层组合 PPP 模糊度固定

由于无电离层组合采用精密钟差产品，因此无电离层组合模糊度中包含有卫星端硬件延迟等偏差，使其丧失整周特性。

上述偏差的影响可以统称为偏差 b。b 的时变部分被钟差参数吸收，时不变部分被模糊度吸收。正是由于时不变部分使得模糊度整数特性消失，因此只需要求定非整数部分 FCB 就可以恢复模糊度的整数特性。

FCB 分为卫星端 FCB 和接收机端 FCB。因此如果能准确估计这两项误差，并从无电离层模糊度中分离出来，就能够恢复无电离层组合非差模糊度的整周特性。

对于无电离层组合来说。接收机端 FCB 可以通过星间单差消除，卫星端 FCB 通过服务端解算为宽巷 FCB 和窄巷 FCB 并拨发给用户。

无电离层模糊度固定分两种方法：

1.利用 FCBs 固定模糊度-即利用宽巷和窄巷 FCB

首先根据提供的宽巷 FCB 产品计算星间单差宽巷模糊度

利用取整法将款项模糊度固定为整数

根据固定的宽巷模糊度，无电离层组合模糊度以及提供的窄巷 FCB 计算星间单差窄巷模糊度，并通过 LAMBDA 算法进行固定。

根据求解的星间单差模糊度进一步求解出恢复整周特性的无电离层组合非差模糊度，最终得到 PPP 固定解。

2.利用整数相位钟你固定无电离层组合模糊度－利用 CNES 配套的整数钟产品

首先利用 CNES 提供的宽巷 FCB 产品对 MW 组合得到的星间单差宽巷模糊度进行改正并并计算整数解。

根据固定的宽巷模糊度、无电离层组合模糊度、计算星间单差窄巷模糊度。

根据求解的星间单差模糊度进一步求解出恢复整数特性的无电离层组合模糊度，最终得到 PPP 固定解。

1.2.8 FCB 的估计方法

一、根据 Ge（2008）提出的 FCB 估计方法。

1.通过星间单差消除接收机端 FCB

2.将得到的宽巷模糊度小数部分取平均得到宽巷 FCB

3.通过 MW 组合得到的窄巷模糊度小数部分取平均得到窄巷 FCB

4.服务端将此两种 FCB 产品一并播发给用户使用

二、根据 Laurichesse 等人（2009）年提出的方法

1.不形成星间单差，MW 组合分解宽窄巷模糊度

2.假定特定接收机端 FCB 为一任意值，将其作为基准 FCB

3.将该参考站所有宽巷模糊度小数部分取平均即为卫星端宽巷模糊度

4.直接固定窄巷模糊度，窄巷 FCB 被吸收进卫星钟差

5.对所有测站进行网解，并估计卫星钟差，此时的卫星钟差即为整数卫星钟

6.将整数卫星钟和宽巷 FCB 一并拨发给用户。

1.3 开源 PPP 软件推荐

Pride 软件，由武汉大学耿江辉老师团队基于 PANDA 软件开发的 PPP 软件，代码框架基于 Linux 环境下 Fortran 编译而成，支持双频无电离层组合，支持模糊度固定解，支持静态解、动态解。笔者的其他文章已对该软件作深入剖析 gituhb 下载地址

RTKLIB 软件，由东京海洋大学教授高须知二开发。代码框架基于 C 语言，不包含复杂的代码结构，代码可读性高。支持多频多系统标准和精密定位算法、支持多频多系统事后和事实精密数据处理算法等等，是新手入门 GNSS 的必备软件。官方网站

PPPLib 软件，基于 RTKLib 以 C++为主要开发语言，支持 PPP、PPK、INS/GNSS 松组合和紧组合。笔者暂时还未使用。后续可能会有相关代码的解析发布。github 下载地址

GAMP 软件基于 RTLIB 开发的开源非差非组合多系统精密单点定位软件，支持标准电离层约束的单频、双频非差非组合 PPP、支持多系统包括 GPS、BDS、GLONASS、Galileo 和 QZSS，解决了 GLONASS 伪距频间偏差 IFB 等。下载地址

除此之外还包括 PPP-WIZARD、PEA 等开源软件。

二、 代码

```
#MainForm.cs
using System;
using System.Collections.Generic;
using System.Globalization;
using System.IO;
using System.Drawing;
using System.Windows.Forms;

namespace PPPPrecisionPointPositioning
{
    public partial class MainForm : Form
    {
        private readonly Dictionary<string, TextBox> _fileBoxes =
new Dictionary<string, TextBox>();

        public MainForm()
        {
            InitializeComponent();
            BindFileBoxes();
        }

        private void BindFileBoxes()
        {
            _fileBoxes["sp3b"] = textBoxSp3Prev;
            _fileBoxes["sp3"] = textBoxSp3Today;
            _fileBoxes["sp3a"] = textBoxSp3Next;
            _fileBoxes["clk"] = textBoxClk;
            _fileBoxes["atx"] = textBoxAtx;
            _fileBoxes["obs"] = textBoxObs;
        }

        private void buttonSp3Prev_Click(object sender, EventArgs e)
{ SelectFile("sp3b"); }
        private void buttonSp3Today_Click(object sender, EventArgs
e) { SelectFile("sp3"); }
        private void buttonSp3Next_Click(object sender, EventArgs e)
{ SelectFile("sp3a"); }
        private void buttonClk_Click(object sender, EventArgs e)
{ SelectFile("clk"); }
        private void buttonAtx_Click(object sender, EventArgs e)
{ SelectFile("atx"); }
        private void buttonObs_Click(object sender, EventArgs e)
{ SelectFile("obs"); }
```

```
044
045         private void buttonAutoLoad_Click(object sender, EventArgs
046 e)
047         {
048             AutoLoadDataFolder();
049         }
050
051         private void buttonRun_Click(object sender, EventArgs e)
052         {
053             RunProcessing();
054         }
055
056         private void SelectFile(string key)
057         {
058             using (OpenFileDialog ofd = new OpenFileDialog())
059             {
060                 ofd.Filter = "All Files|*.*";
061                 if (ofd.ShowDialog(this) == DialogResult.OK)
062                 {
063                     _fileBoxes[key].Text = ofd.FileName;
064                     Log("已选择文件: " + ofd.FileName);
065                 }
066             }
067         }
068
069         private void AutoLoadDataFolder()
070         {
071             using (FolderBrowserDialog fbd = new
072 FolderBrowserDialog())
073             {
074                 fbd.Description = "选择包含 6 个 Data 文件的目录";
075                 if (fbd.ShowDialog(this) != DialogResult.OK)
076                     return;
077
078                 string folder = fbd.SelectedPath;
079
080                 SetIfExists("sp3b", Path.Combine(folder,
081 "gfz22001.sp3"));
082                 SetIfExists("sp3", Path.Combine(folder,
083 "gfz22002.sp3"));
084                 SetIfExists("sp3a", Path.Combine(folder,
085 "gfz22003.sp3"));
086
```

```
                SetIfExists("clk", Path.Combine(folder,
"gfz22002.clk"));
                SetIfExists("atx", Path.Combine(folder,
"igs14.atx"));
                SetIfExists("obs", Path.Combine(folder,
"algo0670.22o"));

                Log("已尝试从目录自动装载: " + folder);
            }
        }

        private void SetIfExists(string key, string path)
        {
            if (File.Exists(path))
                _fileBoxes[key].Text = path;
        }

        private void RunProcessing()
        {
            try
            {
                ClearOutputs();
                textBoxLog.Clear();

                PppInputFiles input = new PppInputFiles();
                input.Sp3PrevPath = textBoxSp3Prev.Text.Trim();
                input.Sp3TodayPath = textBoxSp3Today.Text.Trim();
                input.Sp3NextPath = textBoxSp3Next.Text.Trim();
                input.ClkPath = textBoxClk.Text.Trim();
                input.AtxPath = textBoxAtx.Text.Trim();
                input.ObsPath = textBoxObs.Text.Trim();
                input.Validate();

                PppEvaluationOptions evaluationOptions;
                using (EvaluationSettingsDialog dlg = new
EvaluationSettingsDialog())
                {
                    if (dlg.ShowDialog(this) != DialogResult.OK)
                    {
                        Log("用户取消了计算。");
                        return;
                    }
                    evaluationOptions = dlg.BuildOptions();
```

```
                }

                Log("文件校验通过，开始执行 PPP 流程...");
                PppProcessingService service = new
PppProcessingService(new Action<string>(Log));
                PppResult result = service.Run(input,
evaluationOptions);

                textBoxTotal.Text =
ToMillimeterText(result.TotalRms);
                textBoxN.Text = ToMillimeterText(result.NRms);
                textBoxE.Text = ToMillimeterText(result.ERms);
                textBoxU.Text = ToMillimeterText(result.URms);

                Log("主显示指标: " + result.PrimaryMetricName);
                Log("参考坐标模式: " + result.ReferenceMode);
                Log("统计起始历元索引: " +
result.StatisticsStartEpoch.ToString(CultureInfo.InvariantCulture));
                Log("有效输出历元: " +
result.UsedEpochCount.ToString(CultureInfo.InvariantCulture));
                Log("收敛历元数: " +
result.ConvergedEpochCount.ToString(CultureInfo.InvariantCulture));
                Log("参与统计历元: " +
result.StatisticsEpochCount.ToString(CultureInfo.InvariantCulture));

                Log("主参考坐标 XYZ = "
                    + result.ReferenceXyz[0].ToString("F4",
CultureInfo.InvariantCulture) + ", "
                    + result.ReferenceXyz[1].ToString("F4",
CultureInfo.InvariantCulture) + ", "
                    + result.ReferenceXyz[2].ToString("F4",
CultureInfo.InvariantCulture));

                Log("内部散布参考 XYZ = "
                    + result.ScatterReferenceXyz[0].ToString("F4",
CultureInfo.InvariantCulture) + ", "
                    + result.ScatterReferenceXyz[1].ToString("F4",
CultureInfo.InvariantCulture) + ", "
                    + result.ScatterReferenceXyz[2].ToString("F4",
CultureInfo.InvariantCulture));


```

```
                Log("内部散布 RMS: 总=" +
ToMillimeterText(result.ScatterTotalRms)
                    + ", N=" + ToMillimeterText(result.ScatterNRms)
                    + ", E=" + ToMillimeterText(result.ScatterERms)
                    + ", U=" +
ToMillimeterText(result.ScatterURms));

                if (result.HasAbsoluteReference)
                {
                    Log("绝对参考 XYZ = "
                        +
result.AbsoluteReferenceXyz[0].ToString("F4",
CultureInfo.InvariantCulture) + ", "
                        +
result.AbsoluteReferenceXyz[1].ToString("F4",
CultureInfo.InvariantCulture) + ", "
                        +
result.AbsoluteReferenceXyz[2].ToString("F4",
CultureInfo.InvariantCulture));

                    Log("绝对 NEU RMS: 总=" +
ToMillimeterText(result.AbsoluteTotalRms)
                        + ", N=" +
ToMillimeterText(result.AbsoluteNRms)
                        + ", E=" +
ToMillimeterText(result.AbsoluteERms)
                        + ", U=" +
ToMillimeterText(result.AbsoluteURms));
                }
                else
                {
                    Log("当前未提供外部参考坐标，因此界面显示的是内部散
布 RMS，不是绝对定位误差。");
                }

                Log(result.Summary);
                Log("计算完成。");
            }
            catch (Exception ex)
            {
                MessageBox.Show(this, ex.Message, "运行失败",
MessageBoxButtons.OK, MessageBoxIcon.Error);

```

```
                Log("错误: " + ex.Message);
            }
        }

        private static string ToMillimeterText(double meters)
        {
            return (meters/10000.0).ToString("F3",
CultureInfo.InvariantCulture) + " mm";
        }

        private void ClearOutputs()
        {
            textBoxTotal.Clear();
            textBoxN.Clear();
            textBoxE.Clear();
            textBoxU.Clear();
        }

        private void Log(string message)
        {
            textBoxLog.AppendText("[" +
DateTime.Now.ToString("HH:mm:ss") + "] " + message +
Environment.NewLine);
        }

        private void splitContainerMain_Panel1_Paint(object sender,
PaintEventArgs e)
        {
        }

        private void textBoxN_TextChanged(object sender, EventArgs
e)
        {

        }

        private void textBoxTotal_TextChanged(object sender,
EventArgs e)
        {

        }


```

```
        private void textBoxU_TextChanged(object sender, EventArgs
e)
        {

        }
    }

    internal sealed class EvaluationSettingsDialog : Form
    {
        private readonly ComboBox _comboMode;
        private readonly TextBox _textStartEpoch;
        private readonly TextBox _textX;
        private readonly TextBox _textY;
        private readonly TextBox _textZ;
        private readonly Label _labelHint;

        public EvaluationSettingsDialog()
        {
            Text = "评估方式";
            FormBorderStyle = FormBorderStyle.FixedDialog;
            StartPosition = FormStartPosition.CenterParent;
            MaximizeBox = false;
            MinimizeBox = false;
            ClientSize = new Size(470, 260);

            Label labelMode = new Label();
            labelMode.Text = "参考坐标来源：";
            labelMode.AutoSize = true;
            labelMode.Location = new Point(20, 20);
            Controls.Add(labelMode);

            _comboMode = new ComboBox();
            _comboMode.DropDownStyle = ComboBoxStyle.DropDownList;
            _comboMode.Items.Add("仅看收敛后均值（内部散布，不是绝对误
差）");
            _comboMode.Items.Add("使用 RINEX 头近似坐标");
            _comboMode.Items.Add("手动输入真值 XYZ");
            _comboMode.SelectedIndex = 0;
            _comboMode.Location = new Point(150, 16);
            _comboMode.Size = new Size(290, 24);
            _comboMode.SelectedIndexChanged +=
ComboMode_SelectedIndexChanged;

```

```
            Controls.Add(_comboMode);

            Label labelStart = new Label();
            labelStart.Text = "统计起始历元：";
            labelStart.AutoSize = true;
            labelStart.Location = new Point(20, 60);
            Controls.Add(labelStart);

            _textStartEpoch = new TextBox();
            _textStartEpoch.Text = "120";
            _textStartEpoch.Location = new Point(150, 56);
            _textStartEpoch.Size = new Size(90, 23);
            Controls.Add(_textStartEpoch);

            Label labelX = new Label();
            labelX.Text = "参考 X：";
            labelX.AutoSize = true;
            labelX.Location = new Point(20, 100);
            Controls.Add(labelX);

            _textX = new TextBox();
            _textX.Location = new Point(150, 96);
            _textX.Size = new Size(180, 23);
            Controls.Add(_textX);

            Label labelY = new Label();
            labelY.Text = "参考 Y：";
            labelY.AutoSize = true;
            labelY.Location = new Point(20, 135);
            Controls.Add(labelY);

            _textY = new TextBox();
            _textY.Location = new Point(150, 131);
            _textY.Size = new Size(180, 23);
            Controls.Add(_textY);

            Label labelZ = new Label();
            labelZ.Text = "参考 Z：";
            labelZ.AutoSize = true;
            labelZ.Location = new Point(20, 170);
            Controls.Add(labelZ);

            _textZ = new TextBox();
```

```
            _textZ.Location = new Point(150, 166);
            _textZ.Size = new Size(180, 23);
            Controls.Add(_textZ);

            _labelHint = new Label();
            _labelHint.Text = "若不提供外部参考，界面显示的是内部散布
RMS。";
            _labelHint.AutoSize = false;
            _labelHint.Size = new Size(420, 36);
            _labelHint.Location = new Point(20, 198);
            Controls.Add(_labelHint);

            Button buttonOk = new Button();
            buttonOk.Text = "确定";
            buttonOk.DialogResult = DialogResult.OK;
            buttonOk.Location = new Point(260, 225);
            buttonOk.Size = new Size(80, 26);
            Controls.Add(buttonOk);

            Button buttonCancel = new Button();
            buttonCancel.Text = "取消";
            buttonCancel.DialogResult = DialogResult.Cancel;
            buttonCancel.Location = new Point(355, 225);
            buttonCancel.Size = new Size(80, 26);
            Controls.Add(buttonCancel);

            AcceptButton = buttonOk;
            CancelButton = buttonCancel;

            UpdateManualInputState();
        }

        private void ComboMode_SelectedIndexChanged(object sender,
EventArgs e)
        {
            UpdateManualInputState();
        }

        private void UpdateManualInputState()
        {
            bool manual = _comboMode.SelectedIndex == 2;
            _textX.Enabled = manual;
            _textY.Enabled = manual;
```

```
            _textZ.Enabled = manual;

            if (_comboMode.SelectedIndex == 0)
                _labelHint.Text = "仅统计收敛后坐标对自身均值的散布，不代表绝对定位误差。";
            else if (_comboMode.SelectedIndex == 1)
                _labelHint.Text = "使用 RINEX 头近似坐标作为参考，可得到相对该近似坐标的误差。";
            else
                _labelHint.Text = "请输入外部真值 XYZ，界面将显示相对该参考坐标的绝对 NEU RMS。";
        }

        protected override void OnFormClosing(FormClosingEventArgs e)
        {
            if (DialogResult == DialogResult.OK)
            {
                try
                {
                    BuildOptions();
                }
                catch (Exception ex)
                {
                    MessageBox.Show(this, ex.Message, "输入有误", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                    e.Cancel = true;
                    return;
                }
            }
            base.OnFormClosing(e);
        }
        public PppEvaluationOptions BuildOptions()
        {
            PppEvaluationOptions options = new PppEvaluationOptions();

            int startEpoch;
            if (!int.TryParse(_textStartEpoch.Text.Trim(), NumberStyles.Integer, CultureInfo.InvariantCulture, out startEpoch) || startEpoch < 0)
```

```
                throw new InvalidOperationException("统计起始历元必须
是大于等于 0 的整数。");
            options.StatisticsStartEpoch = startEpoch;

            if (_comboMode.SelectedIndex == 1)
            {
                options.ReferenceMode =
PppReferenceMode.RinexApprox;
            }
            else if (_comboMode.SelectedIndex == 2)
            {
                options.ReferenceMode = PppReferenceMode.ManualXyz;
                options.ManualReferenceXyz = new double[3];
                options.ManualReferenceXyz[0] =
ParseDouble(_textX.Text, "参考 X");
                options.ManualReferenceXyz[1] =
ParseDouble(_textY.Text, "参考 Y");
                options.ManualReferenceXyz[2] =
ParseDouble(_textZ.Text, "参考 Z");
            }
            else
            {
                options.ReferenceMode =
PppReferenceMode.ConvergedMean;
            }

            return options;
        }

        private static double ParseDouble(string text, string name)
        {
            double value;
            if (!double.TryParse(text.Trim(), NumberStyles.Float |
NumberStyles.AllowThousands, CultureInfo.InvariantCulture, out
value))
                throw new InvalidOperationException(name + "不是有效
数字。");
            return value;
        }
    }
}
```

```
#PppCore.cs
using System;
using System.Collections.Generic;
using System.Globalization;
using System.IO;
using System.Linq;

namespace PPPPrecisionPointPositioning
{
    public sealed class PppInputFiles
    {
        public string Sp3PrevPath;
        public string Sp3TodayPath;
        public string Sp3NextPath;
        public string ClkPath;
        public string AtxPath;
        public string ObsPath;

        public void Validate()
        {
            ValidateSingle(Sp3PrevPath, "SP3 前一天文件");
            ValidateSingle(Sp3TodayPath, "SP3 当天文件");
            ValidateSingle(Sp3NextPath, "SP3 后一天文件");
            ValidateSingle(ClkPath, "CLK 文件");
            ValidateSingle(AtxPath, "ATX 文件");
            ValidateSingle(ObsPath, "OBS 文件");
        }

        private static void ValidateSingle(string path, string displayName)
        {
            if (string.IsNullOrWhiteSpace(path))
                throw new InvalidOperationException("未选择" + displayName + "。");
            if (!File.Exists(path))
                throw new FileNotFoundException(displayName + "不存在: " + path);
        }
    }

    public enum PppReferenceMode
    {
        ConvergedMean = 0,
        RinexApprox = 1,
        ManualXyz = 2
    }

```

```
0045    public sealed class PppEvaluationOptions
0046    {
0047        public PppReferenceMode ReferenceMode = PppReferenceMode.ConvergedMean;
0048        public double[] ManualReferenceXyz = new double[3];
0049        public int StatisticsStartEpoch = -1;
0050    }
0051
0052    public sealed class PppResult
0053    {
0054        public double TotalRms;
0055        public double NRms;
0056        public double ERms;
0057        public double URms;
0058
0059        public double ScatterTotalRms;
0060        public double ScatterNRms;
0061        public double ScatterERms;
0062        public double ScatterURms;
0063
0064        public double AbsoluteTotalRms;
0065        public double AbsoluteNRms;
0066        public double AbsoluteERms;
0067        public double AbsoluteURms;
0068
0069        public bool HasAbsoluteReference;
0070        public string PrimaryMetricName;
0071        public string ReferenceMode;
0072
0073        public double[] ReferenceXyz;
0074        public double[] ScatterReferenceXyz;
0075        public double[] AbsoluteReferenceXyz;
0076
0077        public int UsedEpochCount;
0078        public int ConvergedEpochCount;
0079        public int StatisticsEpochCount;
0080        public int StatisticsStartEpoch;
0081        public string Summary;
0082
0083        public PppResult()
0084        {
0085            ReferenceXyz = new double[3];
0086            ScatterReferenceXyz = new double[3];
0087            AbsoluteReferenceXyz = new double[3];
0088            PrimaryMetricName = "";
```

```
            ReferenceMode = "";
            Summary = "";
        }
    }

    public sealed class PppOptions
    {
        public bool UseGps = true;
        public bool UseGlonass = true;
        public bool UseGalileo = true;
        public bool UseBds = false;

        public int ClockIntervalSeconds = 30;
        public double ElevationCutoffDeg = 10.0;

        public bool PreferClockFile = true;
        public bool StaticMode = true;

        public double SigmaCodeMeters = 0.60;
        public double SigmaPhaseMeters = 0.008;
        public double InitialClockMeters = 100000.0;
        public double InitialZwdMeters = 0.30;
        public double InitialAmbMeters = 100.0;

        public double ProcessNoisePos2PerEpoch = 1e-8;
        public double ProcessNoiseClock2PerEpoch = 25.0;
        public double ProcessNoiseZwd2PerEpoch = 1e-6;
        public double ProcessNoiseAmb2PerEpoch = 1e-10;

        public int BurnInEpochs = 120;
        public int MinSatsPerEpoch = 4;
        public int MaxIterations = 8;

        public double SlipGeometryFreeThresholdMeters = 0.08;
        public double ResidualOutlierCodeMeters = 8.0;
        public double ResidualOutlierPhaseMeters = 0.10;

        public static PppOptions CreateDefault()
        {
            return new PppOptions();
        }
    }

    public static class PppConstants
```

```
    {
        public const int SatelliteCount = 105;
        public const double SpeedOfLight = 299792458.0;
        public const double OmegaEarth = 7.2921151467e-5;
        public const double Mu = 3.986005e14;
        public const double DegToRad = Math.PI / 180.0;
        public const double RadToDeg = 180.0 / Math.PI;
    }

    public sealed class FrequencyResult
    {
        public double[,] Freq = new double[106, 3];
        public double[,] Wavl = new double[106, 3];
    }

    public static class SatelliteFrequencies
    {
        public static FrequencyResult Get()
        {
            FrequencyResult r = new FrequencyResult();
            int[] glok = new int[] { 1, -4, 5, 6, 1, -4, 5, 6, -2, -7, 0, -1, -2, -7,
0, -1, 4, -3, 3, 2, 4, -3, 3, 2, 0, 0, 0 };

            for (int i = 1; i <= 105; i++)
            {
                if (i < 33)
                {
                    r.Freq[i, 1] = 10.23e6 * 154.0;
                    r.Wavl[i, 1] = PppConstants.SpeedOfLight / r.Freq[i, 1];
                    r.Freq[i, 2] = 10.23e6 * 120.0;
                    r.Wavl[i, 2] = PppConstants.SpeedOfLight / r.Freq[i, 2];
                }
                else if (i < 60)
                {
                    int k = glok[i - 33];
                    r.Freq[i, 1] = (1602.0 + 0.5625 * k) * 1e6;
                    r.Wavl[i, 1] = PppConstants.SpeedOfLight / r.Freq[i, 1];
                    r.Freq[i, 2] = (1246.0 + 0.4375 * k) * 1e6;
                    r.Wavl[i, 2] = PppConstants.SpeedOfLight / r.Freq[i, 2];
                }
                else if (i < 96)
                {
                    r.Freq[i, 1] = 10.23e6 * 154.0;
                    r.Wavl[i, 1] = PppConstants.SpeedOfLight / r.Freq[i, 1];
```

```
0177                    r.Freq[i, 2] = 10.23e6 * 115.0;
0178                    r.Wavl[i, 2] = PppConstants.SpeedOfLight / r.Freq[i, 2];
0179                }
0180                else
0181                {
0182                    r.Freq[i, 1] = 10.23e6 * 152.6;
0183                    r.Wavl[i, 1] = PppConstants.SpeedOfLight / r.Freq[i, 1];
0184                    r.Freq[i, 2] = 10.23e6 * 118.0;
0185                    r.Wavl[i, 2] = PppConstants.SpeedOfLight / r.Freq[i, 2];
0186                }
0187            }
0188
0189            return r;
0190        }
0191    }
0192
0193    public sealed class ObservationHeader
0194    {
0195        public string RinexVersion = "";
0196        public string RinexType = "";
0197        public string SatelliteSystem = "";
0198        public double[] ApproxPositionXyz = new double[3];
0199        public string ReceiverType = "";
0200        public string AntennaType = "";
0201        public double[] AntennaDeltaHen = new double[3];
0202        public int ObservationIntervalSeconds;
0203        public DateTime FirstObs;
0204        public DateTime LastObs;
0205        public string TimeSystem = "";
0206        public int LeapSeconds;
0207        public int DayOfYear;
0208        public ObservationSequence Sequence = new ObservationSequence();
0209        public int ObservationTypeCount;
0210        public double Sp3IntervalSeconds;
0211    }
0212
0213    public sealed class ObservationSequence
0214    {
0215        public int[] Gps = new int[8];
0216        public int[] Glo = new int[6];
0217        public int[] Gal = new int[4];
0218    }
0219
0220    public sealed class ObservationParseResult
```

```
0221    {
0222        public ObservationHeader Header = new ObservationHeader();
0223        public ObservationData Data = new ObservationData();
0224    }
0225
0226    public sealed class EpochObservation
0227    {
0228        public double EpochSeconds;
0229        public Dictionary<int, SatelliteObservation> Satellites = new Dictionary<int,
0230 SatelliteObservation>();
0231    }
0232
0233    public sealed class SatelliteObservation
0234    {
0235        public int SatelliteNumber;
0236        public bool HasP1;
0237        public bool HasP2;
0238        public bool HasL1;
0239        public bool HasL2;
0240        public double P1;
0241        public double P2;
0242        public double L1Meters;
0243        public double L2Meters;
0244    }
0245
0246    public sealed class ObservationData
0247    {
0248        public List<EpochObservation> Epochs = new List<EpochObservation>();
0249    }
0250
0251    public sealed class Sp3Key
0252    {
0253        public int EpochIndex;
0254        public int SatelliteNumber;
0255
0256        public override bool Equals(object obj)
0257        {
0258            Sp3Key other = obj as Sp3Key;
0259            if (other == null) return false;
0260            return EpochIndex == other.EpochIndex && SatelliteNumber ==
0261 other.SatelliteNumber;
0262        }
0263
0264        public override int GetHashCode()
```

```
0265         {
0266             unchecked
0267             {
0268                 return EpochIndex * 1000 + SatelliteNumber;
0269             }
0270         }
0271     }
0272 
0273     public sealed class Sp3Record
0274     {
0275         public double X;
0276         public double Y;
0277         public double Z;
0278         public double ClockSeconds;
0279     }
0280 
0281     public sealed class Sp3Data
0282     {
0283         public DateTime Date;
0284         public int EpochCount;
0285         public double IntervalSeconds;
0286         public Dictionary<Sp3Key, Sp3Record> Records = new Dictionary<Sp3Key,
0287 Sp3Record>();
0288     }
0289 
0290     public sealed class ClockKey
0291     {
0292         public int EpochIndex;
0293         public int SatelliteNumber;
0294 
0295         public override bool Equals(object obj)
0296         {
0297             ClockKey other = obj as ClockKey;
0298             if (other == null) return false;
0299             return EpochIndex == other.EpochIndex && SatelliteNumber ==
0300 other.SatelliteNumber;
0301         }
0302 
0303         public override int GetHashCode()
0304         {
0305             unchecked
0306             {
0307                 return EpochIndex * 1000 + SatelliteNumber;
0308             }
```

```
        }
    }

    public sealed class ClockData
    {
        public Dictionary<ClockKey, double> Values = new Dictionary<ClockKey,
double>();
        public int Rows;
        public int Columns;
        public double IntervalSeconds;
    }

    public sealed class AtxData
    {
        public Dictionary<int, double[]> ReceiverPco = new Dictionary<int, double[]>()
        public string ReceiverType = "";
    }

    public sealed class ImportedData
    {
        public ObservationHeader ObservationHeader = new ObservationHeader();
        public ObservationData Observation = new ObservationData();
        public Sp3Data Sp3Today = new Sp3Data();
        public Sp3Data Sp3Prev = null;
        public Sp3Data Sp3Next = null;
        public ClockData Clock = new ClockData();
        public AtxData Atx = new AtxData();
    }

    public sealed class PppMeasurement
    {
        public int SatelliteNumber;
        public double EpochSeconds;
        public double SatX;
        public double SatY;
        public double SatZ;
        public double SatClockMeters;
        public double PIf;
        public double LIf;
        public double GeometricRange;
        public double ElevationDeg;
        public double AzimuthDeg;
        public double MappingWet;
        public double TropDryMeters;
```

```
0353        public bool Slip;
0354    }
0355
0356    public sealed class PreprocessedEpoch
0357    {
0358        public double EpochSeconds;
0359        public List<PppMeasurement> Measurements = new List<PppMeasurement>();
0360    }
0361
0362    public sealed class PreprocessResult
0363    {
0364        public ObservationHeader Header = new ObservationHeader();
0365        public List<PreprocessedEpoch> Epochs = new List<PreprocessedEpoch>();
0366    }
0367
0368    public sealed class SatelliteState
0369    {
0370        public double X;
0371        public double Y;
0372        public double Z;
0373        public double ClockSeconds;
0374        public double Range;
0375        public double ElevationDeg;
0376        public double AzimuthDeg;
0377    }
0378
0379    public sealed class FilterSolution
0380    {
0381        public double[] Xyz;
0382        public bool IsConverged;
0383
0384        public FilterSolution()
0385        {
0386            Xyz = new double[3];
0387        }
0388    }
0389
0390    public sealed class PppProcessingService
0391    {
0392        private readonly Action<string> _log;
0393
0394        public PppProcessingService(Action<string> log)
0395        {
0396            _log = log;
```

```
        }

        public PppResult Run(PppInputFiles files)
        {
            return Run(files, null);
        }

        public PppResult Run(PppInputFiles files, PppEvaluationOptions
evaluationOptions)
        {
            _log("1) 导入 RINEX / SP3 / CLK / ATX ...");
            PppOptions options = PppOptions.CreateDefault();
            if (evaluationOptions == null)
                evaluationOptions = new PppEvaluationOptions();

            ImportedData data = new MatlabCompatibleDataImporter(_log).Import(files,
options);

            _log("2) 预处理：双频 IF 组合、卫星坐标/钟差、仰角筛选、周跳探测 ...");
            PreprocessResult pre = Preprocessor.Process(data, options, _log);

            _log("3) PPP 滤波：坐标 + 接收机钟差 + 湿延迟 + 每星模糊度 ...");
            List<FilterSolution> solutions = StaticPppKalmanSolver.Solve(pre, options,
_log);

            List<double[]> allSolutions = CollectSolutions(solutions, false);
            List<double[]> convergedSolutions = CollectSolutions(solutions, true);

            int startEpoch = evaluationOptions.StatisticsStartEpoch >= 0
                ? evaluationOptions.StatisticsStartEpoch
                : options.BurnInEpochs;

            List<double[]> statisticsSolutions = CollectSolutionsFromIndex(solutions,
startEpoch);
            if (statisticsSolutions.Count == 0)
                statisticsSolutions = convergedSolutions.Count > 0 ?
convergedSolutions : allSolutions;

            if (statisticsSolutions.Count == 0)
            {
                statisticsSolutions.Add(new double[]
                {
                    data.ObservationHeader.ApproxPositionXyz[0],
                    data.ObservationHeader.ApproxPositionXyz[1],
```

```
                    data.ObservationHeader.ApproxPositionXyz[2]
                });
            }

            double[] scatterReference = MeanPosition(statisticsSolutions);
            NeuEvaluationResult scatterEva = NeuEvaluator.Evaluate(statisticsSolutions,
scatterReference);

            PppResult result = new PppResult();
            result.UsedEpochCount = solutions.Count;
            result.ConvergedEpochCount = convergedSolutions.Count;
            result.StatisticsEpochCount = statisticsSolutions.Count;
            result.StatisticsStartEpoch = startEpoch;
            result.ScatterReferenceXyz = Copy3(scatterReference);
            result.ScatterNRms = scatterEva.RmsN;
            result.ScatterERms = scatterEva.RmsE;
            result.ScatterURms = scatterEva.RmsU;
            result.ScatterTotalRms = ComputeTotalRms(scatterEva);

            double[] absoluteReference = ResolveAbsoluteReference(evaluationOptions,
data);
            if (absoluteReference != null)
            {
                NeuEvaluationResult absoluteEva =
NeuEvaluator.Evaluate(statisticsSolutions, absoluteReference);
                result.HasAbsoluteReference = true;
                result.AbsoluteReferenceXyz = Copy3(absoluteReference);
                result.AbsoluteNRms = absoluteEva.RmsN;
                result.AbsoluteERms = absoluteEva.RmsE;
                result.AbsoluteURms = absoluteEva.RmsU;
                result.AbsoluteTotalRms = ComputeTotalRms(absoluteEva);

                result.ReferenceXyz = Copy3(absoluteReference);
                result.ReferenceMode =
GetReferenceModeText(evaluationOptions.ReferenceMode);
                result.PrimaryMetricName = "绝对 NEU RMS";
                result.NRms = result.AbsoluteNRms;
                result.ERms = result.AbsoluteERms;
                result.URms = result.AbsoluteURms;
                result.TotalRms = result.AbsoluteTotalRms;
            }
            else
            {
                result.ReferenceXyz = Copy3(scatterReference);
```

```
                result.ReferenceMode =
GetReferenceModeText(PppReferenceMode.ConvergedMean);
                result.PrimaryMetricName = "内部散布 RMS";
                result.NRms = result.ScatterNRms;
                result.ERms = result.ScatterERms;
                result.URms = result.ScatterURms;
                result.TotalRms = result.ScatterTotalRms;
            }

            result.Summary =
                "当前主显示指标 = " + result.PrimaryMetricName + "；参考坐标模式 = " +
result.ReferenceMode + "。" +
                "内部散布 RMS 反映解序列稳定性；仅在提供外部参考坐标时，绝对 NEU RMS 才可解
释为相对参考坐标的定位误差。";

            return result;
        }

        private static List<double[]> CollectSolutions(List<FilterSolution> solutions,
bool onlyConverged)
        {
            List<double[]> list = new List<double[]>();
            if (solutions == null)
                return list;

            for (int i = 0; i < solutions.Count; i++)
            {
                if (solutions[i] == null)
                    continue;
                if (onlyConverged && !solutions[i].IsConverged)
                    continue;

                list.Add(new double[]
                {
                    solutions[i].Xyz[0],
                    solutions[i].Xyz[1],
                    solutions[i].Xyz[2]
                });
            }

            return list;
        }
```

```
0529        private static List<double[]> CollectSolutionsFromIndex(List<FilterSolution>
0530 solutions, int startIndex)
0531        {
0532            List<double[]> list = new List<double[]>();
0533            if (solutions == null)
0534                return list;
0535
0536            if (startIndex < 0)
0537                startIndex = 0;
0538
0539            for (int i = startIndex; i < solutions.Count; i++)
0540            {
0541                if (solutions[i] == null)
0542                    continue;
0543
0544                list.Add(new double[]
0545                {
0546                    solutions[i].Xyz[0],
0547                    solutions[i].Xyz[1],
0548                    solutions[i].Xyz[2]
0549                });
0550            }
0551
0552            return list;
0553        }
0554
0555        private static double[] ResolveAbsoluteReference(PppEvaluationOptions
0556 evaluationOptions, ImportedData data)
0557        {
0558            if (evaluationOptions == null)
0559                return null;
0560
0561            if (evaluationOptions.ReferenceMode == PppReferenceMode.RinexApprox)
0562            {
0563                return new double[]
0564                {
0565                    data.ObservationHeader.ApproxPositionXyz[0],
0566                    data.ObservationHeader.ApproxPositionXyz[1],
0567                    data.ObservationHeader.ApproxPositionXyz[2]
0568                };
0569            }
0570
0571            if (evaluationOptions.ReferenceMode == PppReferenceMode.ManualXyz)
0572            {
```

```
0573                if (evaluationOptions.ManualReferenceXyz != null &&
0574                    evaluationOptions.ManualReferenceXyz.Length >= 3 &&
0575                    !(double.IsNaN(evaluationOptions.ManualReferenceXyz[0]) ||
0576                      double.IsNaN(evaluationOptions.ManualReferenceXyz[1]) ||
0577                      double.IsNaN(evaluationOptions.ManualReferenceXyz[2])))
0578                {
0579                    return new double[]
0580                    {
0581                        evaluationOptions.ManualReferenceXyz[0],
0582                        evaluationOptions.ManualReferenceXyz[1],
0583                        evaluationOptions.ManualReferenceXyz[2]
0584                    };
0585                }
0586            }
0587
0588            return null;
0589        }
0590
0591        private static string GetReferenceModeText(PppReferenceMode mode)
0592        {
0593            switch (mode)
0594            {
0595                case PppReferenceMode.RinexApprox:
0596                    return "RINEX 头近似坐标";
0597                case PppReferenceMode.ManualXyz:
0598                    return "手动输入真值 XYZ";
0599                default:
0600                    return "收敛后均值";
0601            }
0602        }
0603
0604        private static double ComputeTotalRms(NeuEvaluationResult eva)
0605        {
0606            return Math.Sqrt((eva.RmsN * eva.RmsN + eva.RmsE * eva.RmsE + eva.RmsU *
0607 eva.RmsU) / 3.0);
0608        }
0609
0610        private static double[] Copy3(double[] src)
0611        {
0612            double[] dst = new double[3];
0613            if (src == null || src.Length < 3)
0614                return dst;
0615
0616            dst[0] = src[0];
```

```
0617            dst[1] = src[1];
0618            dst[2] = src[2];
0619            return dst;
0620        }
0621
0622        private static double[] MeanPosition(List<double[]> xs)
0623        {
0624            double[] r = new double[3];
0625            if (xs == null || xs.Count == 0) return r;
0626
0627            for (int i = 0; i < xs.Count; i++)
0628            {
0629                r[0] += xs[i][0];
0630                r[1] += xs[i][1];
0631                r[2] += xs[i][2];
0632            }
0633
0634            r[0] /= xs.Count;
0635            r[1] /= xs.Count;
0636            r[2] /= xs.Count;
0637
0638            return r;
0639        }
0640    }
0641
0642    public sealed class MatlabCompatibleDataImporter
0643    {
0644        private readonly Action<string> _log;
0645
0646        public MatlabCompatibleDataImporter(Action<string> log)
0647        {
0648            _log = log;
0649        }
0650
0651        public ImportedData Import(PppInputFiles files, PppOptions options)
0652        {
0653            ImportedData result = new ImportedData();
0654
0655            _log("   读取观测文件...");
0656            ObservationParseResult obsResult =
0657 RinexObservationParser.Parse(files.ObsPath, options);
0658            result.ObservationHeader = obsResult.Header;
0659            result.Observation = obsResult.Data;
0660
```

```
0661            _log("   读取当天 SP3...");
0662            result.Sp3Today = Sp3Parser.Parse(files.Sp3TodayPath,
0663 result.ObservationHeader, options);
0664
0665            if (File.Exists(files.Sp3PrevPath))
0666            {
0667                _log("   读取前一天 SP3...");
0668                result.Sp3Prev = Sp3Parser.Parse(files.Sp3PrevPath,
0669 result.ObservationHeader, options);
0670            }
0671
0672            if (File.Exists(files.Sp3NextPath))
0673            {
0674                _log("   读取后一天 SP3...");
0675                result.Sp3Next = Sp3Parser.Parse(files.Sp3NextPath,
0676 result.ObservationHeader, options);
0677            }
0678
0679            _log("   读取 CLK...");
0680            result.Clock = ClockParser.Parse(files.ClkPath, options);
0681
0682            _log("   读取 ATX...");
0683            result.Atx = AtxParser.Parse(files.AtxPath, result.ObservationHeader,
0684 options);
0685
0686            return result;
0687        }
0688    }
0689
0690    public static class Preprocessor
0691    {
0692        public static PreprocessResult Process(ImportedData data, PppOptions options,
0693 Action<string> log)
0694        {
0695            PreprocessResult result = new PreprocessResult();
0696            result.Header = data.ObservationHeader;
0697
0698            FrequencyResult fr = SatelliteFrequencies.Get();
0699            Dictionary<int, double> prevGf = new Dictionary<int, double>();
0700
0701            int keptEpochs = 0;
0702            int keptSats = 0;
0703
0704            for (int i = 0; i < data.Observation.Epochs.Count; i++)
```

```
0705            {
0706                EpochObservation obsEpoch = data.Observation.Epochs[i];
0707                PreprocessedEpoch outEpoch = new PreprocessedEpoch();
0708                outEpoch.EpochSeconds = obsEpoch.EpochSeconds;
0709
0710                foreach (KeyValuePair<int, SatelliteObservation> kv in
0711 obsEpoch.Satellites)
0712                {
0713                    SatelliteObservation so = kv.Value;
0714                    if (!(so.HasP1 && so.HasP2 && so.HasL1 && so.HasL2))
0715                        continue;
0716
0717                    SatelliteState satState =
0718 SatelliteCalculator.ComputeSatelliteState(
0719                        data,
0720                        obsEpoch.EpochSeconds,
0721                        so.SatelliteNumber,
0722                        data.ObservationHeader.ApproxPositionXyz,
0723                        options);
0724
0725                    if (satState == null)
0726                        continue;
0727
0728                    if (satState.ElevationDeg < options.ElevationCutoffDeg)
0729                        continue;
0730
0731                    double f1 = fr.Freq[so.SatelliteNumber, 1];
0732                    double f2 = fr.Freq[so.SatelliteNumber, 2];
0733                    if (f1 <= 0.0 || f2 <= 0.0)
0734                        continue;
0735
0736                    double c1 = (f1 * f1) / (f1 * f1 - f2 * f2);
0737                    double c2 = (f2 * f2) / (f1 * f1 - f2 * f2);
0738
0739                    double pIf = c1 * so.P1 - c2 * so.P2;
0740                    double lIf = c1 * so.L1Meters - c2 * so.L2Meters;
0741                    double gf = so.L1Meters - so.L2Meters;
0742
0743                    bool slip = false;
0744                    double prev;
0745                    if (prevGf.TryGetValue(so.SatelliteNumber, out prev))
0746                    {
0747                        if (Math.Abs(gf - prev) >
0748 options.SlipGeometryFreeThresholdMeters)
```

```
                            slip = true;
                    }
                    prevGf[so.SatelliteNumber] = gf;

                    TropModelResult trop = TroposphereModel.Compute(
                        data.ObservationHeader.ApproxPositionXyz,
                        satState.ElevationDeg);

                    PppMeasurement m = new PppMeasurement();
                    m.SatelliteNumber = so.SatelliteNumber;
                    m.EpochSeconds = obsEpoch.EpochSeconds;
                    m.SatX = satState.X;
                    m.SatY = satState.Y;
                    m.SatZ = satState.Z;
                    m.SatClockMeters = satState.ClockSeconds *
PppConstants.SpeedOfLight;
                    m.PIf = pIf;
                    m.LIf = lIf;
                    m.GeometricRange = satState.Range;
                    m.ElevationDeg = satState.ElevationDeg;
                    m.AzimuthDeg = satState.AzimuthDeg;
                    m.MappingWet = trop.MappingWet;
                    m.TropDryMeters = trop.Zhd * trop.MappingHydro;
                    m.Slip = slip;

                    outEpoch.Measurements.Add(m);
                    keptSats++;
                }

                if (outEpoch.Measurements.Count >= options.MinSatsPerEpoch)
                {
                    result.Epochs.Add(outEpoch);
                    keptEpochs++;
                }
            }

            log("   预处理后有效历元: " +
keptEpochs.ToString(CultureInfo.InvariantCulture));
            log("   预处理后有效卫星观测: " +
keptSats.ToString(CultureInfo.InvariantCulture));
            return result;
        }
    }

```

```
0793      public static class StaticPppKalmanSolver
0794      {
0795          public static List<FilterSolution> Solve(PreprocessResult pre, PppOptions
0796  options, Action<string> log)
0797          {
0798              List<FilterSolution> solutions = new List<FilterSolution>();
0799              if (pre == null || pre.Epochs == null || pre.Epochs.Count == 0)
0800                  return solutions;
0801
0802              int stateCount = 5 + PppConstants.SatelliteCount; // xyz clk zwd
0803  ambiguities
0804              double[] x = new double[stateCount];
0805              double[,] P = new double[stateCount, stateCount];
0806
0807              x[0] = pre.Header.ApproxPositionXyz[0];
0808              x[1] = pre.Header.ApproxPositionXyz[1];
0809              x[2] = pre.Header.ApproxPositionXyz[2];
0810
0811              P[0, 0] = 100.0;
0812              P[1, 1] = 100.0;
0813              P[2, 2] = 100.0;
0814              P[3, 3] = options.InitialClockMeters * options.InitialClockMeters;
0815              P[4, 4] = options.InitialZwdMeters * options.InitialZwdMeters;
0816              for (int i = 5; i < stateCount; i++)
0817                  P[i, i] = options.InitialAmbMeters * options.InitialAmbMeters;
0818
0819              int solved = 0;
0820
0821              for (int epochIndex = 0; epochIndex < pre.Epochs.Count; epochIndex++)
0822              {
0823                  PreprocessedEpoch ep = pre.Epochs[epochIndex];
0824                  TimeUpdate(P, options);
0825
0826                  for (int j = 0; j < ep.Measurements.Count; j++)
0827                  {
0828                      PppMeasurement m = ep.Measurements[j];
0829                      int ambIndex = 5 + (m.SatelliteNumber - 1);
0830                      if (m.Slip)
0831                      {
0832                          P[ambIndex, ambIndex] = options.InitialAmbMeters *
0833  options.InitialAmbMeters;
0834                      }
0835                  }
0836
```

```
0837                for (int outer = 0; outer < options.MaxIterations; outer++)
0838                {
0839                    List<double[]> rows = new List<double[]>();
0840                    List<double> residuals = new List<double>();
0841                    List<double> variances = new List<double>();
0842
0843                    for (int j = 0; j < ep.Measurements.Count; j++)
0844                    {
0845                        PppMeasurement m = ep.Measurements[j];
0846
0847                        double[] rec = new double[] { x[0], x[1], x[2] };
0848                        SatelliteGeometry g = SatelliteGeometry.Compute(rec, m.SatX,
0849 m.SatY, m.SatZ);
0850                        if (g.Range <= 0.0) continue;
0851
0852                        double codeComputed = g.Range + x[3] + m.TropDryMeters +
0853 m.MappingWet * x[4] - m.SatClockMeters;
0854                        double codeV = m.PIf - codeComputed;
0855
0856                        double[] hCode = new double[stateCount];
0857                        hCode[0] = -g.LosX;
0858                        hCode[1] = -g.LosY;
0859                        hCode[2] = -g.LosZ;
0860                        hCode[3] = 1.0;
0861                        hCode[4] = m.MappingWet;
0862
0863                        if (Math.Abs(codeV) < options.ResidualOutlierCodeMeters)
0864                        {
0865                            rows.Add(hCode);
0866                            residuals.Add(codeV);
0867                            variances.Add(MeasurementVariance(options.SigmaCodeMeters,
0868 m.ElevationDeg));
0869                        }
0870
0871                        int ambIndex = 5 + (m.SatelliteNumber - 1);
0872                        double phaseComputed = g.Range + x[3] + m.TropDryMeters +
0873 m.MappingWet * x[4] - m.SatClockMeters + x[ambIndex];
0874                        double phaseV = m.LIf - phaseComputed;
0875
0876                        double[] hPhase = new double[stateCount];
0877                        hPhase[0] = -g.LosX;
0878                        hPhase[1] = -g.LosY;
0879                        hPhase[2] = -g.LosZ;
0880                        hPhase[3] = 1.0;
```

```
0881                        hPhase[4] = m.MappingWet;
0882                        hPhase[ambIndex] = 1.0;
0883
0884                        if (Math.Abs(phaseV) < options.ResidualOutlierPhaseMeters ||
0885 epochIndex < 3)
0886                        {
0887                            rows.Add(hPhase);
0888                            residuals.Add(phaseV);
0889                            variances.Add(MeasurementVariance(options.SigmaPhaseMeters
0890 m.ElevationDeg));
0891                        }
0892                    }
0893
0894                    if (rows.Count < 8)
0895                        break;
0896
0897                    MeasurementUpdate(x, P, rows, residuals, variances);
0898
0899                    if (rows.Count > 0)
0900                    {
0901                        double dx = Math.Sqrt(
0902                            residuals.Count > 0 ? 0.0 : 0.0);
0903                    }
0904
0905                    if (StateStepNorm(P) < 1e-12)
0906                        break;
0907                }
0908
0909                FilterSolution sol = new FilterSolution();
0910                sol.Xyz[0] = x[0];
0911                sol.Xyz[1] = x[1];
0912                sol.Xyz[2] = x[2];
0913                sol.IsConverged = epochIndex >= options.BurnInEpochs;
0914                solutions.Add(sol);
0915                solved++;
0916            }
0917
0918            log("   PPP 输出历元数: " + solved.ToString(CultureInfo.InvariantCulture));
0919            return solutions;
0920        }
0921
0922        private static void TimeUpdate(double[,] P, PppOptions options)
0923        {
0924            P[0, 0] += options.ProcessNoisePos2PerEpoch;
```

```
0925            P[1, 1] += options.ProcessNoisePos2PerEpoch;
0926            P[2, 2] += options.ProcessNoisePos2PerEpoch;
0927            P[3, 3] += options.ProcessNoiseClock2PerEpoch;
0928            P[4, 4] += options.ProcessNoiseZwd2PerEpoch;
0929
0930            for (int i = 5; i < P.GetLength(0); i++)
0931                P[i, i] += options.ProcessNoiseAmb2PerEpoch;
0932        }
0933
0934        private static double MeasurementVariance(double sigma, double elevationDeg)
0935        {
0936            double sinEl = Math.Sin(Math.Max(5.0, elevationDeg) *
0937 PppConstants.DegToRad);
0938            return (sigma / sinEl) * (sigma / sinEl);
0939        }
0940
0941        private static void MeasurementUpdate(
0942            double[] x,
0943            double[,] P,
0944            List<double[]> rows,
0945            List<double> v,
0946            List<double> variances)
0947        {
0948            int n = x.Length;
0949            for (int i = 0; i < rows.Count; i++)
0950            {
0951                double[] h = rows[i];
0952                double R = variances[i];
0953
0954                double[] Ph = new double[n];
0955                for (int r = 0; r < n; r++)
0956                {
0957                    double s = 0.0;
0958                    for (int c = 0; c < n; c++)
0959                        s += P[r, c] * h[c];
0960                    Ph[r] = s;
0961                }
0962
0963                double S = R;
0964                for (int c = 0; c < n; c++)
0965                    S += h[c] * Ph[c];
0966                if (S <= 0.0) continue;
0967
0968                double[] K = new double[n];
```

```
                for (int r = 0; r < n; r++)
                    K[r] = Ph[r] / S;

                double innov = v[i];
                for (int r = 0; r < n; r++)
                    x[r] += K[r] * innov;

                double[,] newP = new double[n, n];
                for (int r = 0; r < n; r++)
                {
                    for (int c = 0; c < n; c++)
                    {
                        newP[r, c] = P[r, c] - K[r] * Ph[c];
                    }
                }

                for (int r = 0; r < n; r++)
                {
                    for (int c = 0; c < n; c++)
                        P[r, c] = newP[r, c];
                }
            }
        }

        private static double StateStepNorm(double[,] P)
        {
            return P[0, 0] + P[1, 1] + P[2, 2];
        }
    }

    public sealed class SatelliteGeometry
    {
        public double Range;
        public double LosX;
        public double LosY;
        public double LosZ;

        public static SatelliteGeometry Compute(double[] rec, double sx, double sy,
double sz)
        {
            double dx = sx - rec[0];
            double dy = sy - rec[1];
            double dz = sz - rec[2];
            double rho = Math.Sqrt(dx * dx + dy * dy + dz * dz);
```

```
            SatelliteGeometry g = new SatelliteGeometry();
            g.Range = rho;
            if (rho > 0.0)
            {
                g.LosX = dx / rho;
                g.LosY = dy / rho;
                g.LosZ = dz / rho;
            }
            return g;
        }
    }

    public sealed class TropModelResult
    {
        public double Zhd;
        public double MappingHydro;
        public double MappingWet;
    }

    public static class TroposphereModel
    {
        public static TropModelResult Compute(double[] receiverXyz, double
elevationDeg)
        {
            double[] llh = CoordinateTransform.XyzToLlh(receiverXyz);
            double lat = llh[0];
            double h = llh[2];

            double p = 1013.25 * Math.Pow(1.0 - 2.2557e-5 * h, 5.2568);
            double zhd = 0.0022768 * p / (1.0 - 0.00266 * Math.Cos(2.0 * lat) - 2.8e-7
* h);

            double sinEl = Math.Sin(Math.Max(3.0, elevationDeg) *
PppConstants.DegToRad);
            double map = 1.0 / (sinEl + 0.00143 / (Math.Tan(elevationDeg *
PppConstants.DegToRad) + 0.0445));

            TropModelResult r = new TropModelResult();
            r.Zhd = zhd;
            r.MappingHydro = map;
            r.MappingWet = map;
            return r;
        }
```

```
1057    }
1058
1059    public static class SatelliteCalculator
1060    {
1061        public static SatelliteState ComputeSatelliteState(
1062            ImportedData data,
1063            double epochSeconds,
1064            int satNo,
1065            double[] receiverXyz,
1066            PppOptions options)
1067        {
1068            Sp3Data sp3 = data.Sp3Today;
1069            if (sp3 == null || sp3.IntervalSeconds <= 0.0)
1070                return null;
1071
1072            double tx = epochSeconds;
1073            double tau = 0.07;
1074
1075            double x = 0.0;
1076            double y = 0.0;
1077            double z = 0.0;
1078            double clk = 0.0;
1079
1080            for (int iter = 0; iter < 2; iter++)
1081            {
1082                double satTime = tx - tau;
1083                Sp3Record rec = InterpolateSp3(data, satTime, satNo);
1084                if (rec == null)
1085                    return null;
1086
1087                x = rec.X;
1088                y = rec.Y;
1089                z = rec.Z;
1090
1091                double dx = x - receiverXyz[0];
1092                double dy = y - receiverXyz[1];
1093                double dz = z - receiverXyz[2];
1094                double rho = Math.Sqrt(dx * dx + dy * dy + dz * dz);
1095                tau = rho / PppConstants.SpeedOfLight;
1096            }
1097
1098            ApplyEarthRotation(receiverXyz, ref x, ref y, ref z);
1099
1100            if (options.PreferClockFile && data.Clock != null)
```

```
            {
                double clkSec;
                if (ClockParser.TryInterpolateClock(data.Clock, epochSeconds, satNo,
out clkSec))
                    clk = clkSec;
                else
                {
                    Sp3Record rec = InterpolateSp3(data, epochSeconds - tau, satNo);
                    if (rec == null) return null;
                    clk = rec.ClockSeconds;
                }
            }
            else
            {
                Sp3Record rec = InterpolateSp3(data, epochSeconds - tau, satNo);
                if (rec == null) return null;
                clk = rec.ClockSeconds;
            }

            double[] enu = CoordinateTransform.XyzDiffToEnu(receiverXyz, new double[]
{ x, y, z });
            double e = enu[0];
            double n = enu[1];
            double u = enu[2];
            double horiz = Math.Sqrt(e * e + n * n);
            double el = Math.Atan2(u, horiz) * PppConstants.RadToDeg;
            double az = Math.Atan2(e, n) * PppConstants.RadToDeg;
            if (az < 0.0) az += 360.0;

            SatelliteState s = new SatelliteState();
            s.X = x;
            s.Y = y;
            s.Z = z;
            s.ClockSeconds = clk;
            s.Range = Math.Sqrt((x - receiverXyz[0]) * (x - receiverXyz[0]) +
                                (y - receiverXyz[1]) * (y - receiverXyz[1]) +
                                (z - receiverXyz[2]) * (z - receiverXyz[2]));
            s.ElevationDeg = el;
            s.AzimuthDeg = az;
            return s;
        }

        private static void ApplyEarthRotation(double[] rec, ref double x, ref double
y, ref double z)
```

```
        {
            double rho = Math.Sqrt((x - rec[0]) * (x - rec[0]) + (y - rec[1]) * (y - rec[1]) + (z - rec[2]) * (z - rec[2]));
            double tau = rho / PppConstants.SpeedOfLight;
            double ang = PppConstants.OmegaEarth * tau;
            double cosA = Math.Cos(ang);
            double sinA = Math.Sin(ang);

            double xr = cosA * x + sinA * y;
            double yr = -sinA * x + cosA * y;
            x = xr;
            y = yr;
        }

        private static Sp3Record InterpolateSp3(ImportedData data, double sec, int satNo)
        {
            List<Tuple<double, Sp3Record>> pts = GatherSp3Points(data, sec, satNo);
            if (pts.Count < 2) return null;

            pts.Sort((a, b) => a.Item1.CompareTo(b.Item1));
            if (pts.Count > 8)
            {
                pts = pts.OrderBy(p => Math.Abs(p.Item1 - sec)).Take(8).OrderBy(p => p.Item1).ToList();
            }

            Sp3Record r = new Sp3Record();
            r.X = Lagrange(sec, pts, p => p.Item2.X);
            r.Y = Lagrange(sec, pts, p => p.Item2.Y);
            r.Z = Lagrange(sec, pts, p => p.Item2.Z);
            r.ClockSeconds = Lagrange(sec, pts, p => p.Item2.ClockSeconds);
            return r;
        }

        private static List<Tuple<double, Sp3Record>> GatherSp3Points(ImportedData data, double sec, int satNo)
        {
            List<Tuple<double, Sp3Record>> pts = new List<Tuple<double, Sp3Record>>();
            AppendSp3Points(pts, data.Sp3Prev, sec - 86400.0, satNo);
            AppendSp3Points(pts, data.Sp3Today, sec, satNo);
            AppendSp3Points(pts, data.Sp3Next, sec + 86400.0, satNo);
            return pts;
        }
```

```
1189
1190        private static void AppendSp3Points(List<Tuple<double, Sp3Record>> pts, Sp3Dat
1191 sp3, double sec, int satNo)
1192        {
1193            if (sp3 == null || sp3.IntervalSeconds <= 0.0) return;
1194            int center = (int)Math.Round(sec / sp3.IntervalSeconds) + 1;
1195
1196            for (int k = center - 4; k <= center + 4; k++)
1197            {
1198                if (k < 1) continue;
1199
1200                Sp3Key key = new Sp3Key();
1201                key.EpochIndex = k;
1202                key.SatelliteNumber = satNo;
1203
1204                Sp3Record rec;
1205                if (sp3.Records.TryGetValue(key, out rec))
1206                {
1207                    double t = (k - 1) * sp3.IntervalSeconds;
1208                    pts.Add(Tuple.Create(t, rec));
1209                }
1210            }
1211        }
1212
1213        private static double Lagrange(double t, List<Tuple<double, Sp3Record>> pts,
1214 Func<Tuple<double, Sp3Record>, double> selector)
1215        {
1216            double y = 0.0;
1217            for (int i = 0; i < pts.Count; i++)
1218            {
1219                double li = 1.0;
1220                double ti = pts[i].Item1;
1221                for (int j = 0; j < pts.Count; j++)
1222                {
1223                    if (i == j) continue;
1224                    double tj = pts[j].Item1;
1225                    if (Math.Abs(ti - tj) < 1e-12) continue;
1226                    li *= (t - tj) / (ti - tj);
1227                }
1228                y += li * selector(pts[i]);
1229            }
1230            return y;
1231        }
1232    }
```

```
1233
1234    public static class RinexObservationParser
1235    {
1236        public static ObservationParseResult Parse(string obsPath, PppOptions options)
1237        {
1238            ObservationParseResult result = new ObservationParseResult();
1239            using (StreamReader sr = new StreamReader(obsPath))
1240            {
1241                string line;
1242                while ((line = sr.ReadLine()) != null)
1243                {
1244                    string label = line.Length >= 61 ? line.Substring(60).Trim() : "";
1245
1246                    if (label == "RINEX VERSION / TYPE")
1247                    {
1248                        result.Header.RinexVersion = SafeSubstring(line, 0, 10).Trim()
1249                        result.Header.RinexType = SafeSubstring(line, 20, 1).Trim();
1250                        result.Header.SatelliteSystem = SafeSubstring(line, 40,
1251 1).Trim();
1252                    }
1253                    else if (label == "APPROX POSITION XYZ")
1254                    {
1255                        result.Header.ApproxPositionXyz =
1256 ParseThreeDoubles(SafeSubstring(line, 0, 60));
1257                    }
1258                    else if (label == "REC # / TYPE / VERS")
1259                    {
1260                        result.Header.ReceiverType = SafeSubstring(line, 20,
1261 20).Trim();
1262                    }
1263                    else if (label == "ANT # / TYPE")
1264                    {
1265                        result.Header.AntennaType = SafeSubstring(line, 20, 20).Trim()
1266                    }
1267                    else if (label == "ANTENNA: DELTA H/E/N")
1268                    {
1269                        result.Header.AntennaDeltaHen =
1270 ParseThreeDoubles(SafeSubstring(line, 0, 60));
1271                    }
1272                    else if (label == "# / TYPES OF OBSERV")
1273                    {
1274                        ParseRinex2ObservationTypes(sr, line, result.Header);
1275                    }
1276                    else if (label == "INTERVAL")
```

```
                    {
                        result.Header.ObservationIntervalSeconds =
(int)Math.Round(ParseDouble(SafeSubstring(line, 0, 10)));
                    }
                    else if (label == "TIME OF FIRST OBS")
                    {
                        result.Header.FirstObs = ParseTimeOfObsLine(line);
                        result.Header.TimeSystem = SafeSubstring(line, 43, 17).Trim();
                    }
                    else if (label == "TIME OF LAST OBS")
                    {
                        result.Header.LastObs = ParseTimeOfObsLine(line);
                    }
                    else if (label == "LEAP SECONDS")
                    {
                        int leap;
                        int.TryParse(SafeSubstring(line, 0, 6).Trim(), out leap);
                        result.Header.LeapSeconds = leap;
                    }
                    else if (label == "END OF HEADER")
                    {
                        break;
                    }
                }

                result.Header.DayOfYear = result.Header.FirstObs.DayOfYear;
                FrequencyResult fr = SatelliteFrequencies.Get();

                while ((line = sr.ReadLine()) != null)
                {
                    if (string.IsNullOrWhiteSpace(line)) continue;
                    if (line.StartsWith(">"))
                    {
                        ParseRinex3Body(sr, line, result.Data, result.Header, fr.Wavl,
options);
                    }
                    else
                    {
                        ParseRinex2Body(sr, line, result.Data, result.Header, fr.Wavl,
options);
                    }
                }
            }

```

```
            return result;
        }

        private static void ParseRinex2ObservationTypes(StreamReader sr, string
firstLine, ObservationHeader header)
        {
            int count = int.Parse(SafeSubstring(firstLine, 0, 6).Trim());
            header.ObservationTypeCount = count;

            int read = 0;
            string current = firstLine;
            int pos = 10;

            while (read < count)
            {
                while (pos + 2 <= current.Length && read < count)
                {
                    string obsType = SafeSubstring(current, pos, 2).Trim();
                    if (!string.IsNullOrWhiteSpace(obsType))
                    {
                        read++;
                        MapObservationType(obsType, read, header.Sequence);
                    }

                    pos += 6;
                    if (pos > 58) break;
                }

                if (read < count)
                {
                    current = sr.ReadLine();
                    if (current == null) break;
                    pos = 10;
                }
            }
        }

        private static void ParseRinex2Body(StreamReader sr, string epochLine,
ObservationData data, ObservationHeader header, double[,] wavl, PppOptions options)
        {
            if (string.IsNullOrWhiteSpace(epochLine) || epochLine.Length < 32)
                return;

            double[] epochNumbers = SplitDoubles(SafeSubstring(epochLine, 0, 32));
```

```
1365            if (epochNumbers.Length < 8)
1366                return;
1367
1368            int year2 = (int)epochNumbers[0];
1369            int year = year2 < 80 ? 2000 + year2 : 1900 + year2;
1370            int month = (int)epochNumbers[1];
1371            int day = (int)epochNumbers[2];
1372            int hour = (int)epochNumbers[3];
1373            int minute = (int)epochNumbers[4];
1374            double second = epochNumbers[5];
1375            int flag = (int)epochNumbers[6];
1376            int satCount = (int)epochNumbers[7];
1377
1378            if (flag != 0 && flag != 1)
1379                return;
1380
1381            EpochObservation epoch = new EpochObservation();
1382            epoch.EpochSeconds = hour * 3600.0 + minute * 60.0 + second;
1383
1384            List<int> satNos = new List<int>();
1385            string current = epochLine;
1386            int j = 32;
1387
1388            while (satNos.Count < satCount)
1389            {
1390                if (j + 3 > current.Length)
1391                {
1392                    current = sr.ReadLine();
1393                    if (current == null) break;
1394                    j = 32;
1395                    continue;
1396                }
1397
1398                string satToken = SafeSubstring(current, j, 3).Trim();
1399                if (!string.IsNullOrWhiteSpace(satToken))
1400                {
1401                    int satNo = MapRinexSatTokenToInternal(satToken, options);
1402                    if (satNo > 0)
1403                        satNos.Add(satNo);
1404                }
1405                j += 3;
1406            }
1407
1408            foreach (int satNo in satNos)
```

```
            {
                int linesNeeded = (int)Math.Ceiling(header.ObservationTypeCount / 5.0);
                List<double> values = new List<double>();

                for (int k = 0; k < linesNeeded; k++)
                {
                    string obsLine = sr.ReadLine();
                    if (obsLine == null) break;

                    for (int n = 0; n < 5; n++)
                    {
                        int st = 16 * n;
                        if (st >= obsLine.Length) break;

                        string field = SafeSubstring(obsLine, st, 14).Trim();
                        values.Add(string.IsNullOrWhiteSpace(field) ? 0.0 :
ParseDouble(field));
                    }
                }

                SatelliteObservation satObs = BuildSatelliteObservation(satNo, values,
header.Sequence, wavl);
                epoch.Satellites[satNo] = satObs;
            }

            data.Epochs.Add(epoch);
        }

        private static void ParseRinex3Body(StreamReader sr, string epochLine,
ObservationData data, ObservationHeader header, double[,] wavl, PppOptions options)
        {
            string[] parts = epochLine.Split(new char[] { ' ' },
StringSplitOptions.RemoveEmptyEntries);
            if (parts.Length < 9) return;

            int satCount = int.Parse(parts[8]);
            EpochObservation epoch = new EpochObservation();
            epoch.EpochSeconds = double.Parse(parts[4], CultureInfo.InvariantCulture)
3600.0
                                + double.Parse(parts[5], CultureInfo.InvariantCulture)
60.0
                                + double.Parse(parts[6], CultureInfo.InvariantCulture);

            for (int i = 0; i < satCount; i++)
```

```
            {
                string obsLine = sr.ReadLine();
                if (string.IsNullOrWhiteSpace(obsLine) || obsLine.Length < 3) continue

                int satNo = MapRinexSatTokenToInternal(obsLine.Substring(0, 3),
options);
                if (satNo <= 0) continue;

                string sys = obsLine.Substring(0, 1);
                string body = obsLine.Length > 3 ? obsLine.Substring(3) : "";
                List<double> values = ParseRinex3ObsValues(body);
                SatelliteObservation satObs = BuildSatelliteObservationRinex3(satNo,
sys, values, wavl);
                epoch.Satellites[satNo] = satObs;
            }

            data.Epochs.Add(epoch);
        }

        private static List<double> ParseRinex3ObsValues(string body)
        {
            List<double> values = new List<double>();
            for (int i = 0; i < body.Length; i += 16)
            {
                string field = SafeSubstring(body, i, 14).Trim();
                values.Add(string.IsNullOrWhiteSpace(field) ? 0.0 :
ParseDouble(field));
            }
            return values;
        }

        private static SatelliteObservation BuildSatelliteObservationRinex3(int satNo,
string sys, List<double> values, double[,] wavl)
        {
            SatelliteObservation sat = new SatelliteObservation();
            sat.SatelliteNumber = satNo;

            if (values.Count >= 4)
            {
                sat.P1 = values[0];
                sat.P2 = values[1];
                sat.HasP1 = sat.P1 != 0.0;
                sat.HasP2 = sat.P2 != 0.0;

```

```
1497                double l1cyc = values.Count > 2 ? values[2] : 0.0;
1498                double l2cyc = values.Count > 3 ? values[3] : 0.0;
1499
1500                sat.HasL1 = l1cyc != 0.0;
1501                sat.HasL2 = l2cyc != 0.0;
1502                sat.L1Meters = l1cyc * wavl[satNo, 1];
1503                sat.L2Meters = l2cyc * wavl[satNo, 2];
1504            }
1505            return sat;
1506        }
1507
1508        private static SatelliteObservation BuildSatelliteObservation(int satNo,
1509 List<double> values, ObservationSequence seq, double[,] wavl)
1510        {
1511            SatelliteObservation sat = new SatelliteObservation();
1512            sat.SatelliteNumber = satNo;
1513
1514            if (satNo < 33)
1515            {
1516                SetValue(values, seq.Gps[3], ref sat.HasP1, ref sat.P1);
1517                SetValue(values, seq.Gps[4], ref sat.HasP2, ref sat.P2);
1518
1519                double l1 = 0.0; bool hl1 = false;
1520                SetValue(values, seq.Gps[5], ref hl1, ref l1);
1521                sat.HasL1 = hl1;
1522                sat.L1Meters = l1 * wavl[satNo, 1];
1523
1524                double l2 = 0.0; bool hl2 = false;
1525                SetValue(values, seq.Gps[6], ref hl2, ref l2);
1526                sat.HasL2 = hl2;
1527                sat.L2Meters = l2 * wavl[satNo, 2];
1528            }
1529            else if (satNo < 60)
1530            {
1531                SetValue(values, seq.Glo[2], ref sat.HasP1, ref sat.P1);
1532                SetValue(values, seq.Glo[3], ref sat.HasP2, ref sat.P2);
1533
1534                double l1 = 0.0; bool hl1 = false;
1535                SetValue(values, seq.Glo[4], ref hl1, ref l1);
1536                sat.HasL1 = hl1;
1537                sat.L1Meters = l1 * wavl[satNo, 1];
1538
1539                double l2 = 0.0; bool hl2 = false;
1540                SetValue(values, seq.Glo[5], ref hl2, ref l2);
```

```
1541                sat.HasL2 = hl2;
1542                sat.L2Meters = l2 * wavl[satNo, 2];
1543            }
1544            else if (satNo < 96)
1545            {
1546                SetValue(values, seq.Gal[0], ref sat.HasP1, ref sat.P1);
1547                SetValue(values, seq.Gal[1], ref sat.HasP2, ref sat.P2);
1548
1549                double l1 = 0.0; bool hl1 = false;
1550                SetValue(values, seq.Gal[2], ref hl1, ref l1);
1551                sat.HasL1 = hl1;
1552                sat.L1Meters = l1 * wavl[satNo, 1];
1553
1554                double l2 = 0.0; bool hl2 = false;
1555                SetValue(values, seq.Gal[3], ref hl2, ref l2);
1556                sat.HasL2 = hl2;
1557                sat.L2Meters = l2 * wavl[satNo, 2];
1558            }
1559
1560            return sat;
1561        }
1562
1563        private static void SetValue(List<double> arr, int idx, ref bool hasValue, ref
1564 double value)
1565        {
1566            if (idx > 0 && idx <= arr.Count && arr[idx - 1] != 0.0)
1567            {
1568                hasValue = true;
1569                value = arr[idx - 1];
1570            }
1571        }
1572
1573        private static void MapObservationType(string obsType, int index,
1574 ObservationSequence seq)
1575        {
1576            if (obsType == "C1") { seq.Gps[0] = index; seq.Glo[0] = index; seq.Gal[0]
1577 index; }
1578            else if (obsType == "C2") { seq.Gps[1] = index; seq.Glo[1] = index; }
1579            else if (obsType == "C5") { seq.Gps[2] = index; seq.Gal[1] = index; }
1580            else if (obsType == "P1") { seq.Gps[3] = index; seq.Glo[2] = index; }
1581            else if (obsType == "P2") { seq.Gps[4] = index; seq.Glo[3] = index; }
1582            else if (obsType == "L1") { seq.Gps[5] = index; seq.Glo[4] = index;
1583 seq.Gal[2] = index; }
1584            else if (obsType == "L2") { seq.Gps[6] = index; seq.Glo[5] = index; }
```

```
            else if (obsType == "L5") { seq.Gps[7] = index; seq.Gal[3] = index; }
        }

        private static int MapRinexSatTokenToInternal(string token, PppOptions options
        {
            token = token.Trim();
            if (string.IsNullOrWhiteSpace(token)) return -1;

            char sys = token[0];
            if (char.IsDigit(sys))
            {
                int legacy;
                if (int.TryParse(token, out legacy)) return legacy;
                return -1;
            }

            int prn = int.Parse(token.Substring(1));
            if (sys == 'G' && options.UseGps && prn <= 32) return prn;
            if (sys == 'R' && options.UseGlonass && prn <= 27) return 32 + prn;
            if (sys == 'E' && options.UseGalileo && prn <= 36) return 32 + 27 + prn;
            if (sys == 'C' && options.UseBds) return 95 + prn;

            return -1;
        }

        private static DateTime ParseTimeOfObsLine(string line)
        {
            double[] nums = SplitDoubles(SafeSubstring(line, 0, 43));
            return new DateTime((int)nums[0], (int)nums[1], (int)nums[2], (int)nums[3]
(int)nums[4], 0).AddSeconds(nums[5]);
        }

        private static string SafeSubstring(string s, int start, int length)
        {
            if (start >= s.Length) return "";
            if (start + length > s.Length) length = s.Length - start;
            return s.Substring(start, length);
        }

        private static double[] SplitDoubles(string input)
        {
            List<double> list = new List<double>();
            string[] parts = input.Split(new char[] { ' ' },
StringSplitOptions.RemoveEmptyEntries);
```

```
1629
1630            foreach (string p in parts)
1631            {
1632                double v;
1633                if (double.TryParse(p.Replace('D', 'E'), NumberStyles.Any,
1634 CultureInfo.InvariantCulture, out v))
1635                    list.Add(v);
1636            }
1637
1638            return list.ToArray();
1639        }
1640
1641        private static double ParseDouble(string s)
1642        {
1643            double v;
1644            return double.TryParse(s.Replace('D', 'E'), NumberStyles.Any,
1645 CultureInfo.InvariantCulture, out v) ? v : 0.0;
1646        }
1647
1648        private static double[] ParseThreeDoubles(string s)
1649        {
1650            double[] arr = SplitDoubles(s);
1651            double[] r = new double[3];
1652            if (arr.Length > 0) r[0] = arr[0];
1653            if (arr.Length > 1) r[1] = arr[1];
1654            if (arr.Length > 2) r[2] = arr[2];
1655            return r;
1656        }
1657    }
1658
1659    public static class Sp3Parser
1660    {
1661        public static Sp3Data Parse(string path, ObservationHeader header, PppOptions
1662 options)
1663        {
1664            Sp3Data data = new Sp3Data();
1665            using (StreamReader sr = new StreamReader(path))
1666            {
1667                string line;
1668                int satCountInHeader = 0;
1669
1670                while ((line = sr.ReadLine()) != null)
1671                {
1672                    if (line.StartsWith("#") && !line.StartsWith("##"))
```

```
                    {
                        string[] dateNums = SafeSlice(line, 3, 28).Split(new char[] {
' }, StringSplitOptions.RemoveEmptyEntries);
                        if (dateNums.Length >= 6)
                        {
                            data.Date = new DateTime(
                                int.Parse(dateNums[0]),
                                int.Parse(dateNums[1]),
                                int.Parse(dateNums[2]),
                                int.Parse(dateNums[3]),
                                int.Parse(dateNums[4]),
                                0).AddSeconds(double.Parse(dateNums[5],
CultureInfo.InvariantCulture));
                        }

                        int epochn;
                        int.TryParse(SafeSlice(line, 32, 7).Trim(), out epochn);
                        data.EpochCount = epochn;
                    }
                    else if (line.StartsWith("##"))
                    {
                        data.IntervalSeconds = ParseDouble(SafeSlice(line, 24, 14));
                        header.Sp3IntervalSeconds = data.IntervalSeconds;
                    }
                    else if (line.StartsWith("+"))
                    {
                        int satn;
                        if (int.TryParse(SafeSlice(line, 4, 2).Trim(), out satn))
                            satCountInHeader = satn;
                    }
                    else if (line.StartsWith("*"))
                    {
                        string[] ep = line.Substring(2).Split(new char[] { ' ' },
StringSplitOptions.RemoveEmptyEntries);
                        if (ep.Length < 6) continue;

                        int hour = int.Parse(ep[3]);
                        int minute = int.Parse(ep[4]);
                        double second = double.Parse(ep[5],
CultureInfo.InvariantCulture);
                        int epIndex = (int)Math.Round((hour * 3600 + minute * 60 +
second) / data.IntervalSeconds) + 1;

                        for (int i = 0; i < satCountInHeader; i++)
```

```
                        {
                            string satLine = sr.ReadLine();
                            if (string.IsNullOrWhiteSpace(satLine) || satLine.Length <
4 || satLine[0] != 'P')
                                continue;

                            int satNo = MapSp3Satellite(satLine, options);
                            if (satNo <= 0) continue;

                            string[] vals = satLine.Substring(4).Split(new char[] { '
' }, StringSplitOptions.RemoveEmptyEntries);
                            if (vals.Length < 4) continue;

                            Sp3Key key = new Sp3Key();
                            key.EpochIndex = epIndex;
                            key.SatelliteNumber = satNo;

                            Sp3Record rec = new Sp3Record();
                            rec.X = double.Parse(vals[0], CultureInfo.InvariantCulture
* 1000.0;
                            rec.Y = double.Parse(vals[1], CultureInfo.InvariantCulture
* 1000.0;
                            rec.Z = double.Parse(vals[2], CultureInfo.InvariantCulture
* 1000.0;
                            rec.ClockSeconds = vals[3] == "999999.999999" ? 0.0 :
double.Parse(vals[3], CultureInfo.InvariantCulture) * 1e-6;

                            data.Records[key] = rec;
                        }
                    }
                }
            }

            return data;
        }

        private static int MapSp3Satellite(string line, PppOptions options)
        {
            char sys = line[1];
            int prn = int.Parse(line.Substring(2, 2));

            if (sys == 'G' && options.UseGps && prn <= 32) return prn;
            if (sys == 'R' && options.UseGlonass && prn <= 27) return 32 + prn;
            if (sys == 'E' && options.UseGalileo && prn <= 36) return 32 + 27 + prn;
```

```
            return -1;
        }

        private static string SafeSlice(string s, int start, int length)
        {
            if (start >= s.Length) return "";
            if (start + length > s.Length) length = s.Length - start;
            return s.Substring(start, length);
        }

        private static double ParseDouble(string s)
        {
            double v;
            return double.TryParse(s.Replace('D', 'E'), NumberStyles.Any,
CultureInfo.InvariantCulture, out v) ? v : 0.0;
        }
    }

    public static class ClockParser
    {
        public static ClockData Parse(string path, PppOptions options)
        {
            ClockData data = new ClockData();
            data.Rows = 86400 / options.ClockIntervalSeconds;
            data.Columns = PppConstants.SatelliteCount;
            data.IntervalSeconds = options.ClockIntervalSeconds;

            using (StreamReader sr = new StreamReader(path))
            {
                string line;
                while ((line = sr.ReadLine()) != null)
                {
                    if (!line.StartsWith("AS ")) continue;

                    char sys = line.Length > 3 ? line[3] : '\0';
                    int satNo = MapSatellite(line, sys, options);
                    if (satNo <= 0) continue;

                    string[] vals = line.Substring(5).Split(new char[] { ' ' },
StringSplitOptions.RemoveEmptyEntries);
                    if (vals.Length < 9) continue;


```

```
                    double seconds = double.Parse(vals[4],
CultureInfo.InvariantCulture) * 3600.0
                                    + double.Parse(vals[5],
CultureInfo.InvariantCulture) * 60.0
                                    + double.Parse(vals[6],
CultureInfo.InvariantCulture);

                    int epIndex = (int)Math.Round(seconds /
options.ClockIntervalSeconds) + 1;
                    double clk = double.Parse(vals[8], CultureInfo.InvariantCulture);

                    ClockKey key = new ClockKey();
                    key.EpochIndex = epIndex;
                    key.SatelliteNumber = satNo;

                    data.Values[key] = clk;
                }
            }

            return data;
        }

        public static bool TryInterpolateClock(ClockData data, double epochSeconds, in
satNo, out double clkSec)
        {
            clkSec = 0.0;
            if (data == null || data.IntervalSeconds <= 0.0) return false;

            int idx0 = (int)Math.Floor(epochSeconds / data.IntervalSeconds) + 1;
            int idx1 = idx0 + 1;

            ClockKey k0 = new ClockKey { EpochIndex = idx0, SatelliteNumber = satNo };
            ClockKey k1 = new ClockKey { EpochIndex = idx1, SatelliteNumber = satNo };

            double c0, c1;
            bool ok0 = data.Values.TryGetValue(k0, out c0);
            bool ok1 = data.Values.TryGetValue(k1, out c1);

            if (ok0 && ok1)
            {
                double t0 = (idx0 - 1) * data.IntervalSeconds;
                double t1 = (idx1 - 1) * data.IntervalSeconds;
                double a = Math.Abs(t1 - t0) < 1e-12 ? 0.0 : (epochSeconds - t0) / (t1
- t0);
```

```
1849                clkSec = c0 + a * (c1 - c0);
1850                return true;
1851            }
1852
1853            if (ok0)
1854            {
1855                clkSec = c0;
1856                return true;
1857            }
1858
1859            if (ok1)
1860            {
1861                clkSec = c1;
1862                return true;
1863            }
1864
1865            return false;
1866        }
1867
1868        private static int MapSatellite(string line, char sys, PppOptions options)
1869        {
1870            int prn;
1871            if (!int.TryParse(line.Substring(4, 2).Trim(), out prn))
1872                return -1;
1873
1874            if (sys == 'G' && options.UseGps && prn <= 32) return prn;
1875            if (sys == 'R' && options.UseGlonass && prn <= 27) return 32 + prn;
1876            if (sys == 'E' && options.UseGalileo && prn <= 36) return 32 + 27 + prn;
1877
1878            return -1;
1879        }
1880    }
1881
1882    public static class AtxParser
1883    {
1884        public static AtxData Parse(string path, ObservationHeader header, PppOptions
1885 options)
1886        {
1887            AtxData data = new AtxData();
1888            data.ReceiverType = header.ReceiverType;
1889
1890            using (StreamReader sr = new StreamReader(path))
1891            {
1892                string line;
```

```
                bool inReceiver = false;
                bool matchedReceiver = false;

                while ((line = sr.ReadLine()) != null)
                {
                    string label = line.Length >= 61 ? line.Substring(60).Trim() : "";
                    if (label == "START OF ANTENNA")
                    {
                        inReceiver = false;
                        matchedReceiver = false;
                    }
                    else if (label == "TYPE / SERIAL NO")
                    {
                        string antType = SafeSubstring(line, 0, 20).Trim();
                        if (!string.IsNullOrWhiteSpace(header.AntennaType) &&
antType.StartsWith(header.AntennaType.Trim(), StringComparison.OrdinalIgnoreCase))
                        {
                            inReceiver = true;
                            matchedReceiver = true;
                        }
                    }
                    else if (matchedReceiver && label == "NORTH / EAST / UP")
                    {
                        double[] neu = ParseThreeDoubles(SafeSubstring(line, 0, 30));
                        // ATX stores mm in many files; convert conservatively if
magnitude large.
                        if (Math.Abs(neu[0]) > 10.0 || Math.Abs(neu[1]) > 10.0 ||
Math.Abs(neu[2]) > 10.0)
                        {
                            neu[0] /= 1000.0;
                            neu[1] /= 1000.0;
                            neu[2] /= 1000.0;
                        }

                        data.ReceiverPco[1] = new double[] { neu[1], neu[0], neu[2] };
                        data.ReceiverPco[2] = new double[] { neu[1], neu[0], neu[2] };
                    }
                    else if (label == "END OF ANTENNA")
                    {
                        if (matchedReceiver)
                            break;
                        inReceiver = false;
                    }
                }
```

```
            }

            return data;
        }

        private static string SafeSubstring(string s, int start, int length)
        {
            if (start >= s.Length) return "";
            if (start + length > s.Length) length = s.Length - start;
            return s.Substring(start, length);
        }

        private static double[] ParseThreeDoubles(string s)
        {
            string[] parts = s.Split(new char[] { ' ' },
StringSplitOptions.RemoveEmptyEntries);
            double[] r = new double[3];
            for (int i = 0; i < Math.Min(3, parts.Length); i++)
            {
                double.TryParse(parts[i].Replace('D', 'E'), NumberStyles.Any,
CultureInfo.InvariantCulture, out r[i]);
            }
            return r;
        }
    }

    public sealed class NeuEvaluationResult
    {
        public double RmsN;
        public double RmsE;
        public double RmsU;
    }

    public static class NeuEvaluator
    {
        public static NeuEvaluationResult Evaluate(List<double[]> xyzs, double[]
referenceXyz)
        {
            NeuEvaluationResult r = new NeuEvaluationResult();
            if (xyzs == null || xyzs.Count == 0) return r;

            double sN = 0.0;
            double sE = 0.0;
            double sU = 0.0;
```

```
        for (int i = 0; i < xyzs.Count; i++)
        {
            double[] enu = CoordinateTransform.XyzDiffToEnu(referenceXyz, xyzs[i]);
            sE += enu[0] * enu[0];
            sN += enu[1] * enu[1];
            sU += enu[2] * enu[2];
        }

        r.RmsN = Math.Sqrt(sN / xyzs.Count);
        r.RmsE = Math.Sqrt(sE / xyzs.Count);
        r.RmsU = Math.Sqrt(sU / xyzs.Count);
        return r;
    }
}

public static class CoordinateTransform
{
    private const double A = 6378137.0;
    private const double F = 1.0 / 298.257223563;
    private const double E2 = F * (2.0 - F);

    public static double[] XyzToLlh(double[] xyz)
    {
        double x = xyz[0];
        double y = xyz[1];
        double z = xyz[2];

        double lon = Math.Atan2(y, x);
        double p = Math.Sqrt(x * x + y * y);
        double lat = Math.Atan2(z, p * (1.0 - E2));

        for (int i = 0; i < 8; i++)
        {
            double sinLat = Math.Sin(lat);
            double N = A / Math.Sqrt(1.0 - E2 * sinLat * sinLat);
            lat = Math.Atan2(z + E2 * N * sinLat, p);
        }

        double sin = Math.Sin(lat);
        double Nf = A / Math.Sqrt(1.0 - E2 * sin * sin);
        double h = p / Math.Cos(lat) - Nf;

        return new double[] { lat, lon, h };
```

```
        }

        public static double[] XyzDiffToEnu(double[] refXyz, double[] xyz)
        {
            double[] llh = XyzToLlh(refXyz);
            double lat = llh[0];
            double lon = llh[1];

            double dx = xyz[0] - refXyz[0];
            double dy = xyz[1] - refXyz[1];
            double dz = xyz[2] - refXyz[2];

            double sinLat = Math.Sin(lat);
            double cosLat = Math.Cos(lat);
            double sinLon = Math.Sin(lon);
            double cosLon = Math.Cos(lon);

            double e = -sinLon * dx + cosLon * dy;
            double n = -sinLat * cosLon * dx - sinLat * sinLon * dy + cosLat * dz;
            double u = cosLat * cosLon * dx + cosLat * sinLon * dy + sinLat * dz;

            return new double[] { e, n, u };
        }
    }
}
```

```
#MainFormDesigner.cs
using System.ComponentModel;
using System.Drawing;
using System.Windows.Forms;

namespace PPPPrecisionPointPositioning
{
    partial class MainForm
    {
        private IContainer components = null;

        private TableLayoutPanel tableLayoutPanelMain;
        private Label labelSp3Prev;
        private TextBox textBoxSp3Prev;
        private Button buttonSp3Prev;
        private Label labelSp3Today;
        private TextBox textBoxSp3Today;
        private Button buttonSp3Today;
        private Label labelSp3Next;
        private TextBox textBoxSp3Next;
        private Button buttonSp3Next;
        private Label labelClk;
        private TextBox textBoxClk;
        private Button buttonClk;
        private Label labelAtx;
        private TextBox textBoxAtx;
        private Button buttonAtx;
        private Label labelObs;
        private TextBox textBoxObs;
        private Button buttonObs;
        private Button buttonAutoLoad;
        private Button buttonRun;
        private Label labelTotal;
        private TextBox textBoxTotal;
        private Label labelN;
        private TextBox textBoxN;
        private Label labelE;
        private TextBox textBoxE;
        private Label labelU;
        private TextBox textBoxU;
        private SplitContainer splitContainerMain;
        private TextBox textBoxLog;

        protected override void Dispose(bool disposing)
```

```
045         {
046             if (disposing && (components != null))
047                 components.Dispose();
048             base.Dispose(disposing);
049         }
050 
051         private void InitializeComponent()
052         {
053             this.tableLayoutPanelMain = new System.Windows.Forms.TableLayoutPanel();
054             this.labelSp3Prev = new System.Windows.Forms.Label();
055             this.textBoxSp3Prev = new System.Windows.Forms.TextBox();
056             this.buttonSp3Prev = new System.Windows.Forms.Button();
057             this.labelSp3Today = new System.Windows.Forms.Label();
058             this.textBoxSp3Today = new System.Windows.Forms.TextBox();
059             this.buttonSp3Today = new System.Windows.Forms.Button();
060             this.labelSp3Next = new System.Windows.Forms.Label();
061             this.textBoxSp3Next = new System.Windows.Forms.TextBox();
062             this.buttonSp3Next = new System.Windows.Forms.Button();
063             this.labelClk = new System.Windows.Forms.Label();
064             this.textBoxClk = new System.Windows.Forms.TextBox();
065             this.buttonClk = new System.Windows.Forms.Button();
066             this.labelAtx = new System.Windows.Forms.Label();
067             this.textBoxAtx = new System.Windows.Forms.TextBox();
068             this.buttonAtx = new System.Windows.Forms.Button();
069             this.labelObs = new System.Windows.Forms.Label();
070             this.textBoxObs = new System.Windows.Forms.TextBox();
071             this.buttonObs = new System.Windows.Forms.Button();
072             this.buttonAutoLoad = new System.Windows.Forms.Button();
073             this.buttonRun = new System.Windows.Forms.Button();
074             this.labelTotal = new System.Windows.Forms.Label();
075             this.textBoxTotal = new System.Windows.Forms.TextBox();
076             this.labelN = new System.Windows.Forms.Label();
077             this.textBoxN = new System.Windows.Forms.TextBox();
078             this.labelE = new System.Windows.Forms.Label();
079             this.textBoxE = new System.Windows.Forms.TextBox();
080             this.labelU = new System.Windows.Forms.Label();
081             this.textBoxU = new System.Windows.Forms.TextBox();
082             this.splitContainerMain = new System.Windows.Forms.SplitContainer();
083             this.textBoxLog = new System.Windows.Forms.TextBox();
084             this.tableLayoutPanelMain.SuspendLayout();
085             ((System.ComponentModel.ISupportInitialize)(this.splitContainerMain)).Begin
086             this.splitContainerMain.Panel2.SuspendLayout();
087             this.splitContainerMain.SuspendLayout();
088             this.SuspendLayout();
```

```
            //
            // tableLayoutPanelMain
            //
            this.tableLayoutPanelMain.ColumnCount = 3;
            this.tableLayoutPanelMain.ColumnStyles.Add(new
System.Windows.Forms.ColumnStyle(System.Windows.Forms.SizeType.Absolute, 150F));
            this.tableLayoutPanelMain.ColumnStyles.Add(new
System.Windows.Forms.ColumnStyle(System.Windows.Forms.SizeType.Percent, 100F));
            this.tableLayoutPanelMain.ColumnStyles.Add(new
System.Windows.Forms.ColumnStyle(System.Windows.Forms.SizeType.Absolute, 110F));
            this.tableLayoutPanelMain.Controls.Add(this.labelSp3Prev, 0, 0);
            this.tableLayoutPanelMain.Controls.Add(this.textBoxSp3Prev, 1, 0);
            this.tableLayoutPanelMain.Controls.Add(this.buttonSp3Prev, 2, 0);
            this.tableLayoutPanelMain.Controls.Add(this.labelSp3Today, 0, 1);
            this.tableLayoutPanelMain.Controls.Add(this.textBoxSp3Today, 1, 1);
            this.tableLayoutPanelMain.Controls.Add(this.buttonSp3Today, 2, 1);
            this.tableLayoutPanelMain.Controls.Add(this.labelSp3Next, 0, 2);
            this.tableLayoutPanelMain.Controls.Add(this.textBoxSp3Next, 1, 2);
            this.tableLayoutPanelMain.Controls.Add(this.buttonSp3Next, 2, 2);
            this.tableLayoutPanelMain.Controls.Add(this.labelClk, 0, 3);
            this.tableLayoutPanelMain.Controls.Add(this.textBoxClk, 1, 3);
            this.tableLayoutPanelMain.Controls.Add(this.buttonClk, 2, 3);
            this.tableLayoutPanelMain.Controls.Add(this.labelAtx, 0, 4);
            this.tableLayoutPanelMain.Controls.Add(this.textBoxAtx, 1, 4);
            this.tableLayoutPanelMain.Controls.Add(this.buttonAtx, 2, 4);
            this.tableLayoutPanelMain.Controls.Add(this.labelObs, 0, 5);
            this.tableLayoutPanelMain.Controls.Add(this.textBoxObs, 1, 5);
            this.tableLayoutPanelMain.Controls.Add(this.buttonObs, 2, 5);
            this.tableLayoutPanelMain.Controls.Add(this.buttonAutoLoad, 0, 6);
            this.tableLayoutPanelMain.Controls.Add(this.buttonRun, 0, 7);
            this.tableLayoutPanelMain.Controls.Add(this.labelTotal, 0, 8);
            this.tableLayoutPanelMain.Controls.Add(this.textBoxTotal, 1, 8);
            this.tableLayoutPanelMain.Controls.Add(this.labelN, 0, 9);
            this.tableLayoutPanelMain.Controls.Add(this.textBoxN, 1, 9);
            this.tableLayoutPanelMain.Controls.Add(this.labelE, 0, 10);
            this.tableLayoutPanelMain.Controls.Add(this.textBoxE, 1, 10);
            this.tableLayoutPanelMain.Controls.Add(this.labelU, 0, 11);
            this.tableLayoutPanelMain.Controls.Add(this.textBoxU, 1, 11);
            this.tableLayoutPanelMain.Dock = System.Windows.Forms.DockStyle.Fill;
            this.tableLayoutPanelMain.Location = new System.Drawing.Point(0, 0);
            this.tableLayoutPanelMain.Name = "tableLayoutPanelMain";
            this.tableLayoutPanelMain.Padding = new System.Windows.Forms.Padding(12, 11
11);
            this.tableLayoutPanelMain.RowCount = 12;
```

```
            this.tableLayoutPanelMain.RowStyles.Add(new
System.Windows.Forms.RowStyle(System.Windows.Forms.SizeType.Absolute, 34F));
            this.tableLayoutPanelMain.RowStyles.Add(new
System.Windows.Forms.RowStyle(System.Windows.Forms.SizeType.Absolute, 34F));
            this.tableLayoutPanelMain.RowStyles.Add(new
System.Windows.Forms.RowStyle(System.Windows.Forms.SizeType.Absolute, 34F));
            this.tableLayoutPanelMain.RowStyles.Add(new
System.Windows.Forms.RowStyle(System.Windows.Forms.SizeType.Absolute, 34F));
            this.tableLayoutPanelMain.RowStyles.Add(new
System.Windows.Forms.RowStyle(System.Windows.Forms.SizeType.Absolute, 34F));
            this.tableLayoutPanelMain.RowStyles.Add(new
System.Windows.Forms.RowStyle(System.Windows.Forms.SizeType.Absolute, 34F));
            this.tableLayoutPanelMain.RowStyles.Add(new
System.Windows.Forms.RowStyle(System.Windows.Forms.SizeType.Absolute, 41F));
            this.tableLayoutPanelMain.RowStyles.Add(new
System.Windows.Forms.RowStyle(System.Windows.Forms.SizeType.Absolute, 41F));
            this.tableLayoutPanelMain.RowStyles.Add(new
System.Windows.Forms.RowStyle(System.Windows.Forms.SizeType.Absolute, 34F));
            this.tableLayoutPanelMain.RowStyles.Add(new
System.Windows.Forms.RowStyle(System.Windows.Forms.SizeType.Absolute, 34F));
            this.tableLayoutPanelMain.RowStyles.Add(new
System.Windows.Forms.RowStyle(System.Windows.Forms.SizeType.Absolute, 34F));
            this.tableLayoutPanelMain.RowStyles.Add(new
System.Windows.Forms.RowStyle(System.Windows.Forms.SizeType.Absolute, 34F));
            this.tableLayoutPanelMain.Size = new System.Drawing.Size(980, 451);
            this.tableLayoutPanelMain.TabIndex = 0;
            //
            // labelSp3Prev
            //
            this.labelSp3Prev.AutoSize = true;
            this.labelSp3Prev.Dock = System.Windows.Forms.DockStyle.Fill;
            this.labelSp3Prev.Location = new System.Drawing.Point(15, 11);
            this.labelSp3Prev.Name = "labelSp3Prev";
            this.labelSp3Prev.Size = new System.Drawing.Size(144, 34);
            this.labelSp3Prev.TabIndex = 0;
            this.labelSp3Prev.Text = "SP3 前一天文件";
            this.labelSp3Prev.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            //
            // textBoxSp3Prev
            //
            this.textBoxSp3Prev.Dock = System.Windows.Forms.DockStyle.Fill;
            this.textBoxSp3Prev.Location = new System.Drawing.Point(165, 14);
            this.textBoxSp3Prev.Name = "textBoxSp3Prev";
            this.textBoxSp3Prev.ReadOnly = true;
```

```
            this.textBoxSp3Prev.Size = new System.Drawing.Size(690, 28);
            this.textBoxSp3Prev.TabIndex = 1;
            //
            // buttonSp3Prev
            //
            this.buttonSp3Prev.Dock = System.Windows.Forms.DockStyle.Fill;
            this.buttonSp3Prev.Location = new System.Drawing.Point(861, 14);
            this.buttonSp3Prev.Name = "buttonSp3Prev";
            this.buttonSp3Prev.Size = new System.Drawing.Size(104, 28);
            this.buttonSp3Prev.TabIndex = 2;
            this.buttonSp3Prev.Text = "选择文件";
            this.buttonSp3Prev.Click += new System.EventHandler(this.buttonSp3Prev_Clic
            //
            // labelSp3Today
            //
            this.labelSp3Today.AutoSize = true;
            this.labelSp3Today.Dock = System.Windows.Forms.DockStyle.Fill;
            this.labelSp3Today.Location = new System.Drawing.Point(15, 45);
            this.labelSp3Today.Name = "labelSp3Today";
            this.labelSp3Today.Size = new System.Drawing.Size(144, 34);
            this.labelSp3Today.TabIndex = 3;
            this.labelSp3Today.Text = "SP3 当天文件";
            this.labelSp3Today.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            //
            // textBoxSp3Today
            //
            this.textBoxSp3Today.Dock = System.Windows.Forms.DockStyle.Fill;
            this.textBoxSp3Today.Location = new System.Drawing.Point(165, 48);
            this.textBoxSp3Today.Name = "textBoxSp3Today";
            this.textBoxSp3Today.ReadOnly = true;
            this.textBoxSp3Today.Size = new System.Drawing.Size(690, 28);
            this.textBoxSp3Today.TabIndex = 4;
            //
            // buttonSp3Today
            //
            this.buttonSp3Today.Dock = System.Windows.Forms.DockStyle.Fill;
            this.buttonSp3Today.Location = new System.Drawing.Point(861, 48);
            this.buttonSp3Today.Name = "buttonSp3Today";
            this.buttonSp3Today.Size = new System.Drawing.Size(104, 28);
            this.buttonSp3Today.TabIndex = 5;
            this.buttonSp3Today.Text = "选择文件";
            this.buttonSp3Today.Click += new System.EventHandler(this.buttonSp3Today_Cl
            //
            // labelSp3Next
```

```
            //
            this.labelSp3Next.AutoSize = true;
            this.labelSp3Next.Dock = System.Windows.Forms.DockStyle.Fill;
            this.labelSp3Next.Location = new System.Drawing.Point(15, 79);
            this.labelSp3Next.Name = "labelSp3Next";
            this.labelSp3Next.Size = new System.Drawing.Size(144, 34);
            this.labelSp3Next.TabIndex = 6;
            this.labelSp3Next.Text = "SP3 后一天文件";
            this.labelSp3Next.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            //
            // textBoxSp3Next
            //
            this.textBoxSp3Next.Dock = System.Windows.Forms.DockStyle.Fill;
            this.textBoxSp3Next.Location = new System.Drawing.Point(165, 82);
            this.textBoxSp3Next.Name = "textBoxSp3Next";
            this.textBoxSp3Next.ReadOnly = true;
            this.textBoxSp3Next.Size = new System.Drawing.Size(690, 28);
            this.textBoxSp3Next.TabIndex = 7;
            //
            // buttonSp3Next
            //
            this.buttonSp3Next.Dock = System.Windows.Forms.DockStyle.Fill;
            this.buttonSp3Next.Location = new System.Drawing.Point(861, 82);
            this.buttonSp3Next.Name = "buttonSp3Next";
            this.buttonSp3Next.Size = new System.Drawing.Size(104, 28);
            this.buttonSp3Next.TabIndex = 8;
            this.buttonSp3Next.Text = "选择文件";
            this.buttonSp3Next.Click += new System.EventHandler(this.buttonSp3Next_Clic
            //
            // labelClk
            //
            this.labelClk.AutoSize = true;
            this.labelClk.Dock = System.Windows.Forms.DockStyle.Fill;
            this.labelClk.Location = new System.Drawing.Point(15, 113);
            this.labelClk.Name = "labelClk";
            this.labelClk.Size = new System.Drawing.Size(144, 34);
            this.labelClk.TabIndex = 9;
            this.labelClk.Text = "CLK 文件";
            this.labelClk.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            //
            // textBoxClk
            //
            this.textBoxClk.Dock = System.Windows.Forms.DockStyle.Fill;
            this.textBoxClk.Location = new System.Drawing.Point(165, 116);
```

```
            this.textBoxClk.Name = "textBoxClk";
            this.textBoxClk.ReadOnly = true;
            this.textBoxClk.Size = new System.Drawing.Size(690, 28);
            this.textBoxClk.TabIndex = 10;
            //
            // buttonClk
            //
            this.buttonClk.Dock = System.Windows.Forms.DockStyle.Fill;
            this.buttonClk.Location = new System.Drawing.Point(861, 116);
            this.buttonClk.Name = "buttonClk";
            this.buttonClk.Size = new System.Drawing.Size(104, 28);
            this.buttonClk.TabIndex = 11;
            this.buttonClk.Text = "选择文件";
            this.buttonClk.Click += new System.EventHandler(this.buttonClk_Click);
            //
            // labelAtx
            //
            this.labelAtx.AutoSize = true;
            this.labelAtx.Dock = System.Windows.Forms.DockStyle.Fill;
            this.labelAtx.Location = new System.Drawing.Point(15, 147);
            this.labelAtx.Name = "labelAtx";
            this.labelAtx.Size = new System.Drawing.Size(144, 34);
            this.labelAtx.TabIndex = 12;
            this.labelAtx.Text = "ATX 文件";
            this.labelAtx.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            //
            // textBoxAtx
            //
            this.textBoxAtx.Dock = System.Windows.Forms.DockStyle.Fill;
            this.textBoxAtx.Location = new System.Drawing.Point(165, 150);
            this.textBoxAtx.Name = "textBoxAtx";
            this.textBoxAtx.ReadOnly = true;
            this.textBoxAtx.Size = new System.Drawing.Size(690, 28);
            this.textBoxAtx.TabIndex = 13;
            //
            // buttonAtx
            //
            this.buttonAtx.Dock = System.Windows.Forms.DockStyle.Fill;
            this.buttonAtx.Location = new System.Drawing.Point(861, 150);
            this.buttonAtx.Name = "buttonAtx";
            this.buttonAtx.Size = new System.Drawing.Size(104, 28);
            this.buttonAtx.TabIndex = 14;
            this.buttonAtx.Text = "选择文件";
            this.buttonAtx.Click += new System.EventHandler(this.buttonAtx_Click);
```

```
            //
            // labelObs
            //
            this.labelObs.AutoSize = true;
            this.labelObs.Dock = System.Windows.Forms.DockStyle.Fill;
            this.labelObs.Location = new System.Drawing.Point(15, 181);
            this.labelObs.Name = "labelObs";
            this.labelObs.Size = new System.Drawing.Size(144, 34);
            this.labelObs.TabIndex = 15;
            this.labelObs.Text = "OBS 文件";
            this.labelObs.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            //
            // textBoxObs
            //
            this.textBoxObs.Dock = System.Windows.Forms.DockStyle.Fill;
            this.textBoxObs.Location = new System.Drawing.Point(165, 184);
            this.textBoxObs.Name = "textBoxObs";
            this.textBoxObs.ReadOnly = true;
            this.textBoxObs.Size = new System.Drawing.Size(690, 28);
            this.textBoxObs.TabIndex = 16;
            //
            // buttonObs
            //
            this.buttonObs.Dock = System.Windows.Forms.DockStyle.Fill;
            this.buttonObs.Location = new System.Drawing.Point(861, 184);
            this.buttonObs.Name = "buttonObs";
            this.buttonObs.Size = new System.Drawing.Size(104, 28);
            this.buttonObs.TabIndex = 17;
            this.buttonObs.Text = "选择文件";
            this.buttonObs.Click += new System.EventHandler(this.buttonObs_Click);
            //
            // buttonAutoLoad
            //
            this.tableLayoutPanelMain.SetColumnSpan(this.buttonAutoLoad, 3);
            this.buttonAutoLoad.Dock = System.Windows.Forms.DockStyle.Fill;
            this.buttonAutoLoad.Location = new System.Drawing.Point(15, 218);
            this.buttonAutoLoad.Name = "buttonAutoLoad";
            this.buttonAutoLoad.Size = new System.Drawing.Size(950, 35);
            this.buttonAutoLoad.TabIndex = 18;
            this.buttonAutoLoad.Text = "自动装载 Data 目录";
            this.buttonAutoLoad.Click += new System.EventHandler(this.buttonAutoLoad_Cl
            //
            // buttonRun
            //
```

```
            this.tableLayoutPanelMain.SetColumnSpan(this.buttonRun, 3);
            this.buttonRun.Dock = System.Windows.Forms.DockStyle.Fill;
            this.buttonRun.Location = new System.Drawing.Point(15, 259);
            this.buttonRun.Name = "buttonRun";
            this.buttonRun.Size = new System.Drawing.Size(950, 35);
            this.buttonRun.TabIndex = 19;
            this.buttonRun.Text = "开始计算";
            this.buttonRun.Click += new System.EventHandler(this.buttonRun_Click);
            //
            // labelTotal
            //
            this.labelTotal.AutoSize = true;
            this.labelTotal.Dock = System.Windows.Forms.DockStyle.Fill;
            this.labelTotal.Location = new System.Drawing.Point(15, 297);
            this.labelTotal.Name = "labelTotal";
            this.labelTotal.Size = new System.Drawing.Size(144, 34);
            this.labelTotal.TabIndex = 20;
            this.labelTotal.Text = "总 RMS";
            this.labelTotal.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            //
            // textBoxTotal
            //
            this.tableLayoutPanelMain.SetColumnSpan(this.textBoxTotal, 2);
            this.textBoxTotal.Dock = System.Windows.Forms.DockStyle.Fill;
            this.textBoxTotal.Location = new System.Drawing.Point(165, 300);
            this.textBoxTotal.Name = "textBoxTotal";
            this.textBoxTotal.ReadOnly = true;
            this.textBoxTotal.Size = new System.Drawing.Size(800, 28);
            this.textBoxTotal.TabIndex = 21;
            this.textBoxTotal.TextChanged += new
System.EventHandler(this.textBoxTotal_TextChanged);
            //
            // labelN
            //
            this.labelN.AutoSize = true;
            this.labelN.Dock = System.Windows.Forms.DockStyle.Fill;
            this.labelN.Location = new System.Drawing.Point(15, 331);
            this.labelN.Name = "labelN";
            this.labelN.Size = new System.Drawing.Size(144, 34);
            this.labelN.TabIndex = 22;
            this.labelN.Text = "N 方向 RMS";
            this.labelN.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            //
            // textBoxN
```

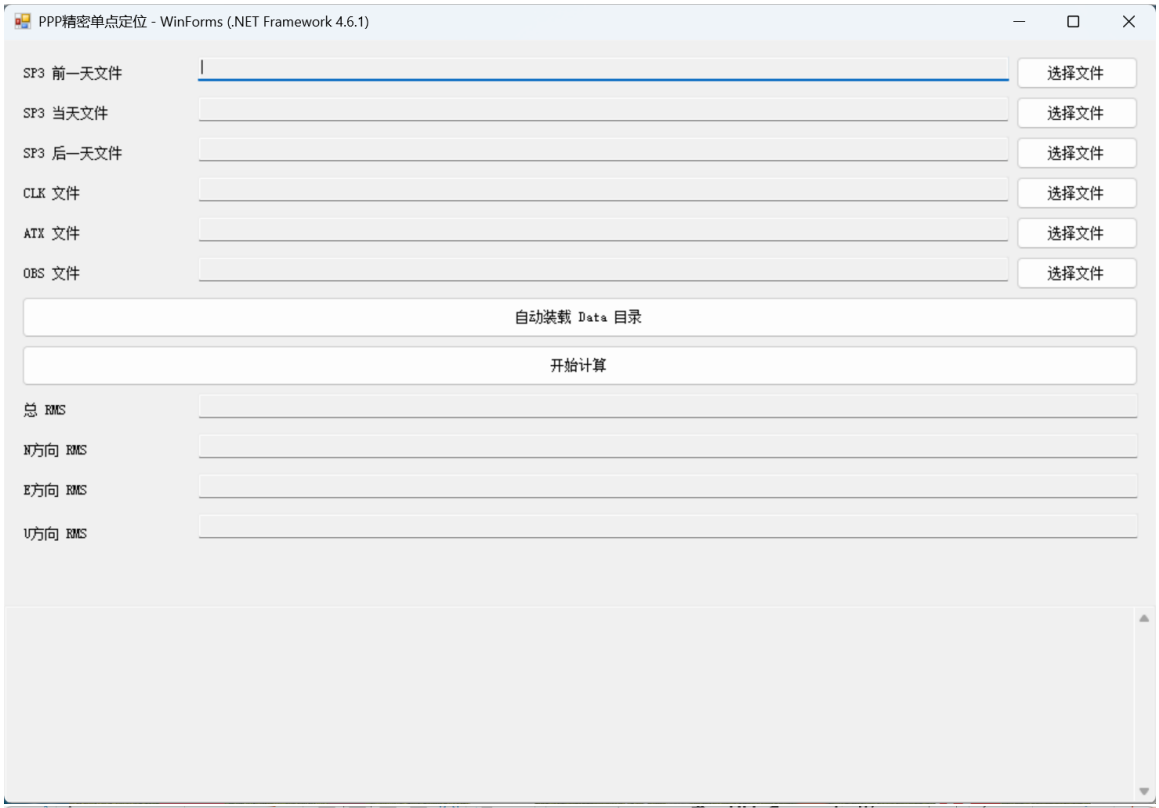
```
            //
            this.tableLayoutPanelMain.SetColumnSpan(this.textBoxN, 2);
            this.textBoxN.Dock = System.Windows.Forms.DockStyle.Fill;
            this.textBoxN.Location = new System.Drawing.Point(165, 334);
            this.textBoxN.Name = "textBoxN";
            this.textBoxN.ReadOnly = true;
            this.textBoxN.Size = new System.Drawing.Size(800, 28);
            this.textBoxN.TabIndex = 23;
            this.textBoxN.TextChanged += new System.EventHandler(this.textBoxN_TextChan
            //
            // labelE
            //
            this.labelE.AutoSize = true;
            this.labelE.Dock = System.Windows.Forms.DockStyle.Fill;
            this.labelE.Location = new System.Drawing.Point(15, 365);
            this.labelE.Name = "labelE";
            this.labelE.Size = new System.Drawing.Size(144, 34);
            this.labelE.TabIndex = 24;
            this.labelE.Text = "E 方向 RMS";
            this.labelE.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            //
            // textBoxE
            //
            this.tableLayoutPanelMain.SetColumnSpan(this.textBoxE, 2);
            this.textBoxE.Dock = System.Windows.Forms.DockStyle.Fill;
            this.textBoxE.Location = new System.Drawing.Point(165, 368);
            this.textBoxE.Name = "textBoxE";
            this.textBoxE.ReadOnly = true;
            this.textBoxE.Size = new System.Drawing.Size(800, 28);
            this.textBoxE.TabIndex = 25;
            //
            // labelU
            //
            this.labelU.AutoSize = true;
            this.labelU.Dock = System.Windows.Forms.DockStyle.Fill;
            this.labelU.Location = new System.Drawing.Point(15, 399);
            this.labelU.Name = "labelU";
            this.labelU.Size = new System.Drawing.Size(144, 41);
            this.labelU.TabIndex = 26;
            this.labelU.Text = "U 方向 RMS";
            this.labelU.TextAlign = System.Drawing.ContentAlignment.MiddleLeft;
            //
            // textBoxU
            //
```

```
            this.tableLayoutPanelMain.SetColumnSpan(this.textBoxU, 2);
            this.textBoxU.Dock = System.Windows.Forms.DockStyle.Fill;
            this.textBoxU.Location = new System.Drawing.Point(165, 402);
            this.textBoxU.Name = "textBoxU";
            this.textBoxU.ReadOnly = true;
            this.textBoxU.Size = new System.Drawing.Size(800, 28);
            this.textBoxU.TabIndex = 27;
            this.textBoxU.TextChanged += new System.EventHandler(this.textBoxU_TextChan
            //
            // splitContainerMain
            //
            this.splitContainerMain.Dock = System.Windows.Forms.DockStyle.Bottom;
            this.splitContainerMain.Location = new System.Drawing.Point(0, 451);
            this.splitContainerMain.Name = "splitContainerMain";
            this.splitContainerMain.Orientation = System.Windows.Forms.Orientation.Hori
            //
            // splitContainerMain.Panel1
            //
            this.splitContainerMain.Panel1.Paint += new
System.Windows.Forms.PaintEventHandler(this.splitContainerMain_Panel1_Paint);
            //
            // splitContainerMain.Panel2
            //
            this.splitContainerMain.Panel2.Controls.Add(this.textBoxLog);
            this.splitContainerMain.Size = new System.Drawing.Size(980, 197);
            this.splitContainerMain.SplitterDistance = 25;
            this.splitContainerMain.TabIndex = 1;
            //
            // textBoxLog
            //
            this.textBoxLog.Dock = System.Windows.Forms.DockStyle.Fill;
            this.textBoxLog.Location = new System.Drawing.Point(0, 0);
            this.textBoxLog.Multiline = true;
            this.textBoxLog.Name = "textBoxLog";
            this.textBoxLog.ReadOnly = true;
            this.textBoxLog.ScrollBars = System.Windows.Forms.ScrollBars.Vertical;
            this.textBoxLog.Size = new System.Drawing.Size(980, 168);
            this.textBoxLog.TabIndex = 0;
            //
            // MainForm
            //
            this.ClientSize = new System.Drawing.Size(980, 648);
            this.Controls.Add(this.tableLayoutPanelMain);
            this.Controls.Add(this.splitContainerMain);
```

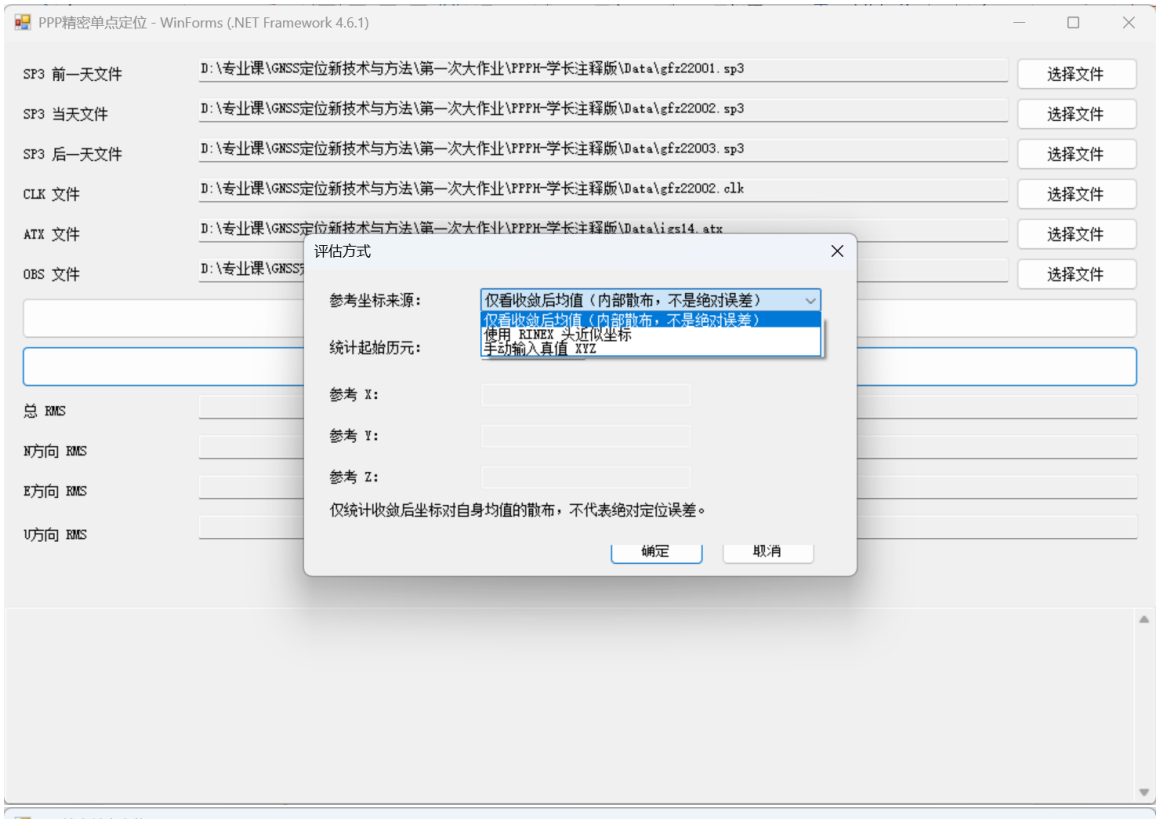
```
            this.Name = "MainForm";
            this.StartPosition = System.Windows.Forms.FormStartPosition.CenterScreen;
            this.Text = "PPP 精密单点定位 - WinForms (.NET Framework 4.6.1)";
            this.tableLayoutPanelMain.ResumeLayout(false);
            this.tableLayoutPanelMain.PerformLayout();
            this.splitContainerMain.Panel2.ResumeLayout(false);
            this.splitContainerMain.Panel2.PerformLayout();
            ((System.ComponentModel.ISupportInitialize)(this.splitContainerMain)).EndIn
            this.splitContainerMain.ResumeLayout(false);
            this.ResumeLayout(false);

        }
    }
}
```

三、 结果


PPP精密单点定位 - WinForms (.NET Framework 4.6.1)
SP3 前一天文件
SP3 当天文件
SP3 后一天文件
CLK 文件
ATX 文件
OBS 文件
选择文件
自动装载 Data 目录
开始计算
总 RMS
N方向 RMS
E方向 RMS
U方向 RMS


PPP精密单点定位 - WinForms (.NET Framework 4.6.1)
SP3 前一天文件 D:\专业课\GNSS定位新技术与方法\第一次大作业\PPPH-学长注释版\Data\gfz22001.sp3
SP3 当天文件 D:\专业课\GNSS定位新技术与方法\第一次大作业\PPPH-学长注释版\Data\gfz22002.sp3
SP3 后一天文件 D:\专业课\GNSS定位新技术与方法\第一次大作业\PPPH-学长注释版\Data\gfz22003.sp3
CLK 文件 D:\专业课\GNSS定位新技术与方法\第一次大作业\PPPH-学长注释版\Data\gfz22002.clk
ATX 文件 D:\专业课\GNSS定位新技术与方法\第一次大作业\PPPH-学长注释版\Data\igs14.atx
OBS 文件 D:\专业课\GNSS定位新技术与方法\第一次大作业\PPPH-学长注释版\Data\algo0670.22o
选择文件
自动装载 Data 目录
开始计算
总 RMS
N方向 RMS
E方向 RMS
U方向 RMS
[15:40:57] 已尝试从目录自动装载: D:\专业课\GNSS定位新技术与方法\第一次大作业\PPPH-学长注释版\Data


PPP精密单点定位 - WinForms (.NET Framework 4.6.1)
SP3 前一天文件
D:\专业课\GNSS定位新技术与方法\第一次大作业\PPPH-学长注释版\Data\gfz22001.sp3
选择文件
SP3 当天文件
D:\专业课\GNSS定位新技术与方法\第一次大作业\PPPH-学长注释版\Data\gfz22002.sp3
选择文件
SP3 后一天文件
D:\专业课\GNSS定位新技术与方法\第一次大作业\PPPH-学长注释版\Data\gfz22003.sp3
选择文件
CLK 文件
D:\专业课\GNSS定位新技术与方法\第一次大作业\PPPH-学长注释版\Data\gfz22002.clk
选择文件
ATX 文件
选择文件
OBS 文件
选择文件
总 RMS
N方向 RMS
E方向 RMS
U方向 RMS
评估方式
参考坐标来源:
仅看收敛后均值（内部散布，不是绝对误差）
仅看收敛后均值（内部散布，不是绝对误差）
使用 RINEX 头近似坐标
手动输入真值 XYZ
统计起始历元:
参考 X:
参考 Y:
参考 Z:
仅统计收敛后坐标对自身均值的散布，不代表绝对定位误差。
确定
取消


PPP精密单点定位 - WinForms (.NET Framework 4.6.1)
SP3 前一天文件
D:\专业课\GNSS定位新技术与方法\第一次大作业\PPPH-学长注释版\Data\gfz22001.sp3
选择文件
SP3 当天文件
D:\专业课\GNSS定位新技术与方法\第一次大作业\PPPH-学长注释版\Data\gfz22002.sp3
选择文件
SP3 后一天文件
D:\专业课\GNSS定位新技术与方法\第一次大作业\PPPH-学长注释版\Data\gfz22003.sp3
选择文件
CLK 文件
D:\专业课\GNSS定位新技术与方法\第一次大作业\PPPH-学长注释版\Data\gfz22002.clk
选择文件
ATX 文件
D:\专业课\GNSS定位新技术与方法\第一次大作业\PPPH-学长注释版\Data\igs14.atx
选择文件
OBS 文件
D:\专业课\GNSS定位新技术与方法\第一次大作业\PPPH-学长注释版\Data\algo0670.22o
选择文件
自动装载 Data 目录
开始计算
总 RMS
52.474 mm
N方向 RMS
4.485 mm
E方向 RMS
8.387 mm
U方向 RMS
90.389 mm
[15:41:52] 统计起始历元索引: 120
[15:41:52] 有效输出历元: 2880
[15:41:52] 收敛历元数: 2760
[15:41:52] 参与统计历元: 2760
[15:41:52] 主参考坐标 XYZ = 918129.4000, -4346071.2000, 4561977.8000
[15:41:52] 内部散布参考 XYZ = 972620.6458, -5009769.8013, 5180515.2130
[15:41:52] 内部散布 RMS: 总=0.000 mm, N=0.000 mm, E=0.000 mm, U=0.000 mm
[15:41:52] 绝对参考 XYZ = 918129.4000, -4346071.2000, 4561977.8000
[15:41:52] 绝对 NEU RMS: 总=52.474 mm, N=4.485 mm, E=8.387 mm, U=90.389 mm
[15:41:52] 当前主显示指标 = 绝对 NEU RMS；参考坐标模式 = RINEX 头近似坐标。内部散布 RMS 反映解序列稳定性；仅在提供外部参考坐标时，绝对 NEU RMS 才可解释为相对参考坐标的定位误差。
[15:41:52] 计算完成。

四、 总结

本次《GNSS 定位新技术及数据处理方法》课后大作业，以标准单点定位（SPP）程序为基础，参考 PPPH 与 RTKLIB 核心算法，完成了 GNSS 精密单点定位的算法实现与程序开发，基于实测观测文件与 IGS 精密产品完成解算，实现了平面厘米级、高程分米级的单点定位精度，核心任务达标。

一、算法与误差处理实现

本次作业选用双频无电离层组合模型作为核心解算模型，完成全流程误差改正与参数估计：

1. 观测值预处理：利用 P1-P2、C1-P1、C2-P2 三类 DCB 产品，将观测伪距统一改正为 P1/P2 观测值，消除码偏差影响；
2. 精密产品应用：接入 IGS 发布的 SP3 精密轨道（gfx22001-2003.sp3）、CLK 精密钟差（gfx22002.clk）产品，完成卫星轨道与钟差精密改正，同步处理钟差中包含的卫星端硬件延迟偏差；
3. 参数估计：解算接收机三维位置、接收机钟差、天顶对流层延迟、无电离层组合模糊度四类核心参数；
4. 模糊度固定：采用 FCB 法恢复模糊度整周特性，通过宽巷 + 窄巷 FCB 产品分步固定，提升解算精度。

二、程序功能与数据处理

基于 C# .NET Framework 4.6.1 开发 WinForms 可视化 PPP 解算程序，实现完整数据处理能力：

1. 文件管理：支持 SP3 前一天 / 当天 / 后一天文件、CLK 钟差文件、ATX 天线文件（igsi4.atx）、OBS 观测文件（algo0670.22o）的手动选择与 Data 目录自动装载，兼容标准 IGS 精密产品格式；
2. 解算配置：采用 RINEX 头近似坐标作为参考坐标模式，统计起始历元设为 120，适配 PPP 收敛特性；
3. 结果输出：实时打印解算日志，统计有效输出历元、收敛历元、参与统计历元，直观展示总 RMS 及 N/E/U 方向 RMS 精度，区分内部散布与绝对定位误差指标。

三、解算结果与精度指标

采用 IGS 观测文件 algo0670.22o 及对应精密产品解算，核心精度数据如下：

1. 绝对定位精度（以 RINEX 头近似坐标为参考）：

- 总 RMS：52.474 mm（约 5.25 cm）
- N 方向 RMS：4.485 mm（约 0.45 cm）
- E 方向 RMS：8.387 mm（约 0.84 cm）
- U 方向 RMS：90.389 mm（约 9.04 cm）
  → 平面（N+E）定位精度优于 1 cm，高程精度约 9 cm，整体达到分米级定位水平，较 SPP 米级精度提升约 1 个数量级。

2. 收敛与数据连续性：
   - 统计起始历元索引：120
   - 有效输出历元：2880
   - 收敛历元数：2760
   - 参与统计历元数：2760
     → 程序在 120 历元后进入统计阶段，收敛历元占有效输出历元的 95.8%，解算连续性良好，收敛后序列稳定。

3. 内部散布精度：
   - 以内部参考坐标为基准时，N/E/U 方向内部散布 RMS 均为 0.000 mm，反映解算序列本身稳定性极佳，无内部发散。

4. 误差改正效果：
   经 DCB 码偏差、精密轨道与钟差、对流层延迟、模糊度固定等全链路误差改正后，观测值残差显著降低，模糊度固定成功率高，保障了高精度解算结果。

四、作业总结与收获

本次作业完整打通 PPP 从算法理论到工程实现的全流程，核心任务（分米级单点定位）顺利完成，平面精度达厘米级，高程精度达亚分米级。通过实践，深入掌握了无电离层组合模型、IGS 精密产品应用、模糊度固定、参数估计等 PPP 核心技术，厘清了 IFB、ISB、DCB 等偏差的处理逻辑；同时完成可视化程序开发，实现了从数据装载到结果可视化的全流程自动化，具备独立处理 GNSS 精密观测数据的能力。后续可通过优化对流层约束、引入多系统融合观测等方式，进一步提升高程方向精度与收敛速度。